



Universidad Internacional de la Rioja (UNIR)

Escuela Superior de Ingeniería y
Tecnología

Máster Universitario en Inteligencia Artificial

Predicción de Cadenas de
Ataque en Infraestructu-
ras Kubernetes mediante
Graph Neural Networks

Trabajo Fin de Estudios

presentado por: Milton Obed Rayo Viana

Dirigido por: Pedro León Millán

Ciudad: Medellín

Fecha: Mayo de 2026

Resumen

Este trabajo presenta *kubescan*, un sistema de detección automática de cadenas de ataque en infraestructuras *Kubernetes* que combina un clasificador *Random Forest* (Capa 1) con una Red Neuronal de Grafos de Atención — *Graph Attention Network* (GAT) — (Capa 2) y optimización de pesos mediante Algoritmo Genético (Capa 3).

A partir de 2.827 manifiestos de *Kubernetes* procedentes de cinco fuentes públicas complementarias, se construyen 599 grafos de clúster originales (1.154 tras aumentación de datos) con aristas que representan relaciones de privilegio, movimiento lateral, proximidad, espacio de nombres y RBAC. La Capa 1 alcanza una F1 macro de 0,9946 en la clasificación de manifiestos individuales, con $AUC = 0,9986$ y cero falsos negativos en el conjunto de test. La Capa 2, basada en GAT de 3 capas con 4 cabezas de atención, *embedding* de tipos de arista y aumentación de grafos, logra F1 macro = $0,803 \pm 0,137$ y Precisión@5 = $0,720 \pm 0,160$ en validación cruzada de 5 pliegues con un protocolo de particionado consciente de grupos y de familias de plantilla (sin fuga entre variantes casi duplicadas y sus grafos base), superando el objetivo de diseño de $P@5 \geq 0,70$; a través de tres semillas de entrenamiento la media es de $0,68 \pm 0,03$, con el objetivo dentro de una desviación típica. En el conjunto de test —86 grafos con 5 cadenas de ataque de repositorios reales, excluido de la validación cruzada y del ajuste del *ensemble*— el sistema completo de tres capas, combinado con una restricción de viabilidad estructural (un clúster de un solo manifiesto no puede albergar una cadena multi-salto), alcanza Precisión@1 = 1,00 y Precisión@3 = 1,00, recupera 4 de las 5 cadenas en las primeras 5 posiciones (Precisión@5 = 0,80) y no produce falsos positivos de clústeres limpios ($FPR_{clean} = 0\%$). Los resultados, acompañados de intervalos de confianza acordes al tamaño del corpus, incluyen además una contribución metodológica: la detección y corrección de una fuga de información por familias de plantilla que inflaba las métricas de evaluaciones anteriores, junto con la ablación de función de pérdida y criterio de selección de *checkpoints* que explica y revierte la aparente regresión asociada.

Palabras clave: Kubernetes, Graph Neural Networks, Random Forest, cadenas de ataque, infraestructura como código.

Abstract

This work presents *kubescan*, an automatic attack-chain detection system for *Kubernetes* infrastructures that combines a Random Forest classifier (Layer 1) with a Graph Attention Network (Graph Attention Network (Red de Atención sobre Grafos) (GAT)) (Layer 2) and Genetic Algorithm ensemble weight optimisation (Layer 3).

From 2,827 *Kubernetes* manifests drawn from five complementary public sources, 599 original cluster graphs are built (1,154 after data augmentation), with edges representing privilege-escalation, lateral-movement, proximity, namespace, and Role-Based Access Control (Control de Acceso Basado en Roles) (RBAC) relationships. Layer 1 achieves macro-F1 = 0.9946, Area Under the Curve (Área Bajo la Curva ROC) (AUC) = 0.9986, and zero false negatives on the test set. Layer 2, a 3-layer GAT with 4 attention heads, edge-type embeddings and graph augmentation, achieves macro-F1 = 0.803 ± 0.137 and Precision@5 = 0.720 ± 0.160 under a 5-fold cross-validation protocol that is group- and template-family-aware (no leakage between near-duplicate variants and their base clusters), exceeding the P@5 ≥ 0.70 design target; across three training seeds the mean is 0.68 ± 0.03 , with the target within one standard deviation. On the test set — 86 graphs containing 5 attack chains from real-world repositories, held out from cross-validation and ensemble tuning — the full three-layer system, combined with a structural feasibility constraint (a single-manifest cluster cannot host a multi-hop chain), achieves Precision@1 = 1.00 and Precision@3 = 1.00, recovers 4 of the 5 attack chains within the top-5 ranking (Precision@5 = 0.80), and produces no false positives on clean clusters (FPR_{clean} = 0). Results, reported with confidence intervals appropriate to the corpus size, additionally include a methodological contribution: the detection and correction of template-family information leakage that inflated earlier evaluations, together with the loss and checkpoint-selection ablation that explains and reverses the apparent regression associated with it.

Keywords: *Kubernetes*, Graph Neural Networks, Random Forest, attack chains, infrastructure as code.

Índice de Contenidos

Resumen	I
Abstract	II
1 Introducción	1
1.1 Motivación	1
1.2 Planteamiento del problema	2
1.3 Objetivos generales	2
1.4 Estructura del trabajo	2
2 Estado del Arte	4
2.1 Seguridad en infraestructuras Kubernetes	4
2.2 Análisis estático de IaC	5
2.3 Aprendizaje automático para detección de vulnerabilidades	6
2.4 Redes neuronales de grafos	9
2.5 GNNs aplicadas a ciberseguridad	10
2.6 Detección de cadenas de ataque y grafos de ataque en Kubernetes	11
2.7 Síntesis y posicionamiento	12
3 Objetivos y Metodología	14
3.1 Objetivo general	14
3.2 Objetivos específicos	14
3.3 Metodología	15
3.4 Estrategia de partición y validación de datos	16
3.5 Métricas de evaluación	17
4 Diseño e Implementación	19
4.1 Identificación de requisitos	19

4.1.1	Requisitos funcionales	19
4.1.2	Requisitos no funcionales	20
4.2	Arquitectura general del sistema	20
4.3	Construcción del dataset	21
4.3.1	Fuentes de datos	21
4.3.2	Características extraídas	22
4.4	Construcción del grafo de clúster	23
4.4.1	Detección de escape y movimiento lateral	24
4.4.2	Detección de privilegio RBAC	25
4.5	Capa 1: Random Forest	25
4.5.1	Modelo binario	25
4.5.2	Modelo de severidad	26
4.6	Capa 2: Graph Attention Network	26
4.6.1	Arquitectura	26
4.6.2	Entrenamiento	27
4.6.3	Aumentación de datos	28
4.7	Capa 3: Ensemble con Algoritmo Genético	29
4.7.1	Restricción de viabilidad estructural	30
4.8	Implementación de kubescan	31
4.9	Tabla de características del sistema	31
4.10	Calidad de software y proceso de validación	31
5	Resultados y Evaluación	35
5.1	Resultados de la Capa 1: Random Forest	35
5.1.1	Clasificación binaria	35
5.1.2	Importancia de características	36
5.1.3	Modelo de severidad	37
5.2	Resultados de la Capa 2: GNN	38
5.2.1	Evolución por fase experimental	38
5.2.2	Análisis de Precisión@5	40
5.2.3	Resultados por pliegue (fase final)	41
5.2.4	Ablación de función de pérdida y criterio de selección	41
5.2.5	Robustez frente a la semilla de entrenamiento	42

5.3	Resultados de la Capa 3: Ensemble	43
5.4	Análisis de la clasificación por clases: GNN	46
5.5	Validación con herramientas de análisis estático	47
5.6	Evaluación funcional de la herramienta	48
5.7	Comparación con el estado del arte	48
5.7.1	Capa 1 en contexto	49
5.7.2	Capa 2 en contexto	49
6	Conclusiones y Trabajo Futuro	51
6.1	Conclusiones	51
6.2	Consideraciones éticas y uso responsable	53
6.3	Limitaciones	54
6.3.1	Amenazas a la validez	57
6.4	Trabajo futuro	58
	Referencias bibliográficas	61
A	Repositorio de Código y Datos	65
B	Guía de Uso y Reproducción	66
B.1	Instalación	66
B.2	Uso de la interfaz de línea de comandos	66
B.2.1	Análisis de un directorio de manifiestos	66
B.2.2	Análisis de un clúster en ejecución	69
B.3	Reproducción del pipeline de investigación	69
C	Acrónimos y Abreviaturas	71

Índice de Ilustraciones

4.1	Arquitectura del sistema <i>kubescan</i> : pipeline de tres capas desde manifiestos YAML Ain't Markup Language (YAML) de entrada hasta la clasificación de riesgo del clúster. Los <i>risk_scores</i> producidos por la Capa 1 se incorporan como atributo nodal 26 del grafo de entrada de la Capa 2.	21
4.2	Ejemplo de grafo de clúster Kubernetes con cuatro nodos, mostrando los tres tipos de nodo (escape, movimiento lateral y limpio) y las aristas dirigidas que cada uno origina.	24
5.1	Matriz de confusión del modelo Random Forest en el conjunto de test (566 muestras). Los 3 falsos positivos son manifiestos seguros con alta densidad de características de riesgo; no existe ningún falso negativo.	36
5.2	Importancia de las 10 características más relevantes del clasificador Random Forest. Las dos primeras (en rojo) concentran el 48 % de la capacidad discriminativa total del modelo.	37
5.3	Evolución de F1 macro y Precisión@5 de la Capa 2 a lo largo de las cuatro primeras fases experimentales (protocolo preliminar).	40
5.4	Precisión@k del ensemble sobre el conjunto de test (86 grafos, 5 cadenas, restricción de viabilidad estructural).	45
B.1	Flujo de ejecución de <i>kubescan scan</i> sobre el ejemplo <i>produccion-web</i> , desde la invocación por Command-Line Interface (Interfaz de Línea de Comandos) (CLI) hasta el veredicto final.	67

Índice de Tablas

2.1	Comparativa de algoritmos de ML para clasificación de manifiestos Infrastructure as Code (Infraestructura como Código) (IaC)	8
2.2	Comparativa de herramientas de seguridad para Kubernetes	13
4.1	Composición del conjunto de datos de manifiestos Kubernetes	22
4.2	Tipos de aristas en el grafo de clúster	23
4.3	Características del clasificador Random Forest (RF) — grupo Rahman (1/2)	32
4.4	Características del clasificador RF — grupos derivado y extendido, y nodo (2/2)	33
5.1	Resultados del Random Forest — clasificación binaria	35
5.2	F1 por clase — modelo de severidad (3 clases, test)	38
5.3	Evolución de resultados de la Capa 2 (GAT, 5-fold Cross-Validation (Validación Cruzada) (CV)). Las fases 1–4 emplean el protocolo de particionado preliminar (con fuga por variantes aumentadas) y se reportan como registro histórico; la fase 5 introduce el protocolo <i>group-aware</i> sobre el corpus preliminar; la fase 6 es el resultado final, con protocolo <i>group-aware</i> sobre el corpus completo.	39
5.4	Resultados por pliegue — Capa 2, fase final (particionado consciente de familias sobre el corpus completo; los pliegues particionan 513 grafos originales, 32 cadenas; test excluido)	41
5.5	Ablación 2×2 de función de pérdida × criterio de selección de <i>checkpoints</i> (5-fold CV, corpus completo, particionado por familias, semilla 42)	42
5.6	Robustez multi-semilla de la configuración final (particionado fijo; test con restricción de viabilidad estructural, Sección 4.7.1)	43

5.7	Ablación de componentes del ensemble en el conjunto de test (86 grafos, 5 cadenas, techo $P@5 = 1,00$; restricción de viabilidad estructural aplicada en todas las configuraciones)	46
5.8	Ablación de arquitectura — Capa 2 (5-fold CV, corpus completo, particionado por familias, misma configuración que la fase final)	47

Índice de Ecuaciones

2.1	Actualización de la representación de un nodo en una capa GNN	9
2.2	Coeficiente de atención en Graph Attention Networks	9
4.1	Función de puntuación del ensemble (Capa 3)	21
4.2	Cálculo de pesos de clase para el entrenamiento de la GAT	28
4.3	Función objetivo (fitness) del Algoritmo Genético	29

1. Introducción

1.1. Motivación

La adopción masiva de *Kubernetes* como plataforma de orquestación de contenedores ha transformado la manera en que las organizaciones despliegan y gestionan sus aplicaciones. La Cloud Native Computing Foundation (CNCF) reporta que el 96 % de las organizaciones utilizan o evalúan *Kubernetes* (Cloud Native Computing Foundation, 2022), y los informes sectoriales de seguridad documentan que la mayoría de las organizaciones ha sufrido al menos un incidente de seguridad en sus entornos de contenedores durante el último año (Red Hat, 2024). Sin embargo, esta proliferación ha ampliado considerablemente la superficie de ataque: una sola misconfiguration en un manifiesto YAML puede habilitar la escalada de privilegios desde un contenedor comprometido hasta el nodo anfitrión del clúster o, en casos extremos, hasta la infraestructura subyacente completa (National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022).

Los enfoques de análisis estático tradicionales —herramientas como *Checkov* (Bridgecrew, 2021) o *kube-linter* (StackRox, 2021)— detectan misconfiguraciones aisladas en manifiestos individuales, pero no modelan las relaciones entre recursos del clúster. Una única flag de privilegio en un pod no es suficiente para determinar si existe una cadena de ataque viable; es la combinación de nodos de escape con recursos de movimiento lateral la que habilita rutas de explotación completas, siguiendo la lógica de encadenamiento de etapas del modelo de *kill chain* de Hutchins, Cloppert, y Amin 2011; las técnicas concretas de escalada de privilegios en contenedores están catalogadas en el marco MITRE ATT&CK for Containers (MITRE Corporation y Center for Threat-Informed Defense, 2021).

Herramientas más recientes como *KubeHound* (Datadog Security Labs, 2023) e *IceKube* (WithSecure Labs, 2023) abordan este problema construyendo grafos de ataque sobre clústeres activos. Sin embargo, requieren acceso con privilegios al servidor API de *Kubernetes*, lo que limita su aplicabilidad como control preventivo en el pipeline de Continuous Integration / Continuous Delivery (Integración y Entrega Continua) (CI/CD) —un principio conocido como *shift-left security*: adelantar los controles de seguridad a las etapas más tempranas del desarrollo.

Este trabajo aborda ese vacío: construir un sistema capaz de razonar sobre la *estructura* del clúster a partir de manifiestos YAML estáticos, sin requerir un clúster activo, para

identificar automáticamente cadenas de ataque potenciales antes de que sean desplegadas.

1.2. Planteamiento del problema

Formalmente, dado un conjunto de manifiestos YAML que describen un clúster *Kubernetes*, el problema consiste en:

1. Clasificar cada manifiesto individual como seguro o misconfigured (Capa 1).
2. Modelar el clúster como un grafo heterogéneo y clasificarlo en una de tres categorías: *clean* (sin misconfiguraciones relevantes), *isolated* (misconfiguraciones sin cadena viable) o *attack_chain* (existe una ruta de escalada de privilegios hasta movimiento lateral) (Capa 2).
3. Combinar las señales de ambas capas mediante una función de puntuación optimizada para maximizar la Precisión@5 —la fracción de clústeres de cadena de ataque en el top-5 del ranking— minimizando falsos positivos (Capa 3).

La restricción de operar sobre manifiestos YAML estáticos —sin requerir acceso al clúster activo— es un requisito de diseño explícito que diferencia el presente trabajo de las soluciones de grafo de ataque existentes y lo posiciona como un control de seguridad integrable en pipelines CI/CD.

1.3. Objetivos generales

El objetivo general es diseñar, implementar y evaluar un sistema de detección de cadenas de ataque en *Kubernetes* que complemente los enfoques de análisis estático por manifiesto individual con el contexto estructural del clúster, mediante la combinación de aprendizaje automático clásico y aprendizaje sobre grafos, operando principalmente sobre manifiestos YAML estáticos, si bien incorpora opcionalmente una vía de consulta directa a un clúster activo (Capítulo 3).

1.4. Estructura del trabajo

El trabajo se organiza como sigue. El Capítulo 2 revisa el estado del arte en seguridad de *Kubernetes*, herramientas de análisis estático y grafos de ataque, redes neuronales de

grafos y detección de cadenas de ataque; incluye una tabla comparativa de las herramientas más relevantes y un análisis del posicionamiento de *kubescan*. El Capítulo 3 define los objetivos específicos y la metodología de cinco fases. El Capítulo 4 describe el diseño e implementación del sistema, incluyendo el análisis de requisitos, la arquitectura de tres capas, la construcción del dataset y el grafo, y la tabla completa de las 28 características del clasificador. El Capítulo 5 presenta los resultados experimentales de cada capa, la validación cruzada con una herramienta de auditoría externa (*Checkov*) y la evaluación funcional de la herramienta. El Capítulo 6 recoge las conclusiones vinculadas a cada objetivo, las limitaciones, las consideraciones éticas y las líneas de trabajo futuro.

2. Estado del Arte

La detección automática de vulnerabilidades en infraestructuras *Kubernetes* se encuentra en la intersección de cuatro áreas de investigación activa: el análisis de seguridad en infraestructura como código (*Infrastructure as Code*, IaC), el aprendizaje automático aplicado a la ciberseguridad, las redes neuronales de grafos para razonamiento estructural, y la construcción de grafos de ataque para la identificación de rutas de explotación. A continuación se revisan los trabajos y herramientas más relevantes en cada área, y se posiciona la contribución de este trabajo respecto a ellos.

2.1. Seguridad en infraestructuras Kubernetes

Kubernetes es la plataforma de orquestación de contenedores dominante en entornos de producción empresariales, con un 96 % de organizaciones que la utilizan o evalúan según la encuesta anual de la CNCF (Cloud Native Computing Foundation, 2022). A pesar de su adopción generalizada, la seguridad sigue siendo uno de los principales retos operacionales: el informe *State of Kubernetes Security* de Red Hat 2024 (Red Hat, 2024) revela que el 89 % de las organizaciones sufrió al menos un incidente de seguridad relacionado con *Kubernetes* en el último año, y que el 67 % retrasó despliegues a producción por preocupaciones de seguridad. El 46 % reportó pérdidas de clientes o ingresos directamente atribuibles a incidentes en entornos de contenedores. Entre los riesgos más temidos, las vulnerabilidades representan el 33 % de las preocupaciones de los equipos, mientras que las misconfiguraciones y exposiciones alcanzan el 27 %.

Su modelo declarativo, basado en ficheros YAML que especifican el estado deseado del clúster, ofrece grandes ventajas en reproducibilidad y automatización, pero también introduce una amplia superficie de ataque: un fichero mal configurado desplegado en producción puede conceder privilegios excesivos a un contenedor o exponer secretos en texto plano. La flexibilidad de *Kubernetes* —que permite configurar desde el sistema de ficheros del anfitrión hasta los permisos de red a nivel de pod— implica que el número de vectores de ataque crece proporcionalmente con la riqueza del modelo de configuración.

La Agencia de Seguridad Nacional de Estados Unidos (NSA) y la Agencia de Ciberseguridad e Infraestructura (CISA) publicaron en 2022 una guía técnica (National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022) que enumera las mis-

configuraciones más críticas: uso de `privileged: true`, montaje del socket de Docker, compartición del espacio de nombres de proceso del anfitrión (*hostPID*), y automontaje de tokens de cuenta de servicio con permisos excesivos. Estos vectores son precisamente los que este trabajo modeliza como nodos de escape en el grafo del clúster.

Los trabajos de BishopFox (Art, 2021) demostraron empíricamente que un único pod con configuración *everything-allowed* puede comprometer el nodo anfitrión en segundos y pivotar lateralmente a través del clúster. La plataforma educativa *Kubernetes Goat* (Akula, 2020) sistematiza más de veinte escenarios de vulnerabilidad con los que se pueden reproducir rutas de explotación completas en un entorno controlado. Ambas fuentes son la base de las etiquetas de cadena de ataque del presente trabajo.

El marco taxonómico de referencia para los ataques en entornos de contenedores y *Kubernetes* es *MITRE ATT&CK for Containers* (MITRE Corporation y Center for Threat-Informed Defense, 2021), incorporado en la versión 9 de la base de conocimiento ATT&CK. Este marco describe las tácticas y técnicas empleadas por adversarios a nivel de orquestador (*Kubernetes*) y a nivel de contenedor (*Docker*), incluyendo las que este trabajo detecta directamente: escape de contenedor hacia el nodo anfitrión (T1611, *Escape to Host*), movimiento lateral mediante el uso de *tokens* de cuentas de servicio (T1550.001, *Use Alternate Authentication Material: Application Access Token*) y abuso de cuentas válidas (T1078, *Valid Accounts*). La alineación entre las *escape flags* del presente sistema y las técnicas ATT&CK valida que el espacio de detección cubre los vectores de ataque documentados y más explotados en la industria.

2.2. Análisis estático de IaC

El análisis estático de manifiestos IaC cuenta con una larga trayectoria en la literatura. Rahman et al. (Rahman, Shamim, Bose, y Pandita, 2023) construyeron un conjunto de datos de referencia para manifiestos *Kubernetes* extraídos de repositorios públicos, con 2.039 manifiestos de 92 repositorios etiquetados en 11 categorías de *misconfiguration* mediante su propia herramienta de análisis estático. De ese trabajo, el presente sistema recupera y codifica 1.906 manifiestos de GitHub y GitLab como vectores de características binarias que indican la presencia o ausencia de cada patrón.

Las herramientas industriales actuales más utilizadas son *Checkov* (Bridgecrew, 2021), que verifica más de mil políticas de seguridad predefinidas, y *kube-linter* (StackRox, 2021),

especializado en buenas prácticas de *Kubernetes*. Ambas operan al nivel de manifiesto individual y no detectan patrones distribuidos entre varios recursos del clúster. También destaca *Kubescape* (ARMO Ltd., 2021), una plataforma de seguridad de código abierto integrada en la CNCF que combina análisis estático con validación de cumplimiento contra los marcos National Security Agency (NSA), MITRE ATT&CK y CIS *Benchmarks*. Al igual que Checkov y kube-linter, sin embargo, Kubescape evalúa recursos de forma independiente y no construye ningún modelo del clúster como estructura relacional.

La noción de *Unified Misconfiguration Index* (UMI), propuesta por Malul, Meidan, Mimran, Elovici, y Shabtai 2024 en *GenKubeSec*, normaliza y unifica las etiquetas de múltiples herramientas de análisis basadas en reglas para poder compararlas, un problema afín al de generar un *risk_score* unificado como el de la Capa 1 del presente sistema. Su propuesta, que incorpora un modelo de lenguaje de gran escala (LLM) para la clasificación, es complementaria a la de este trabajo, aunque opera en el mismo nivel de manifiesto individual.

Una limitación estructural de todas las herramientas anteriores es que operan sobre manifiestos en aislamiento: saben que un pod tiene `privileged: true`, pero no saben si ese pod tiene acceso de red a los recursos críticos del clúster. La detección de cadenas de ataque requiere, inevitablemente, razonar sobre la estructura del clúster como un todo.

2.3. Aprendizaje automático para detección de vulnerabilidades

El aprendizaje automático se ha aplicado a la detección de vulnerabilidades en código fuente y configuraciones desde hace más de una década. Los clasificadores basados en *Random Forest* (Breiman, 2001) han demostrado resultados robustos en escenarios de alta dimensionalidad y clases desbalanceadas —características típicas de los conjuntos de datos de seguridad— gracias a su capacidad de combinar árboles de decisión con ponderación de clases y selección aleatoria de características (*max_features=sqrt*). El proceso de selección de hiperparámetros para *Random Forest* en tareas de seguridad sigue generalmente la estrategia de búsqueda en rejilla (*grid search*) con validación cruzada estratificada, explorando el espacio de `n_estimators` (100–1.000), `max_depth` (sin límite vs. 10–20) y `min_samples_leaf` (1–5), y seleccionando la combinación que maximiza la métrica objetivo sobre los pliegues de validación (Pedregosa y cols., 2011).

En el dominio de la detección de intrusiones en red, el conjunto de datos UNSW-NB15 (Moustafa y Slay, 2015) se ha convertido en un referente de evaluación para clasificadores supervisados. El conjunto de manifiestos de este trabajo presenta un desequilibrio de clases moderado (56,3 % seguros vs. 43,7 % *misconfigured*); el desafío de desequilibrio más severo se concentra en las *flags* de escape individuales, algunas presentes en menos del 1 % de los manifiestos reales. El desequilibrio de clases es un problema generalizado en los datasets de seguridad: las misconfiguraciones críticas son rarísimas en repositorios reales, lo que obliga a adoptar estrategias explícitas de compensación. La alternativa más común es asignar pesos de clase inversamente proporcionales a su frecuencia (*class_weight=balanced* en *scikit-learn*), que es la estrategia adoptada en este trabajo (Pedregosa y cols., 2011). Otras alternativas como SMOTE (*Synthetic Minority Over-sampling Technique*) no se consideraron apropiadas dado que las características son binarias, y la interpolación lineal entre muestras binarias puede generar vectores fuera del espacio semántico válido.

Trabajos más recientes han explorado el uso de modelos de lenguaje de gran escala (Large Language Model (Modelo de Lenguaje de Gran Escala) (LLM)s) para la detección y remediación de misconfiguraciones en *Kubernetes* (Malul y cols., 2024). Si bien los LLMs ofrecen capacidades de razonamiento semántico superiores, requieren recursos computacionales sustancialmente mayores, producen salidas no deterministas y son más opacos que los clasificadores clásicos, lo que dificulta su interpretación en auditorías de seguridad donde la trazabilidad de las reglas de detección es un requisito normativo. Para entornos de producción donde el operador de seguridad debe poder justificar ante una auditoría por qué un clúster fue marcado como de alto riesgo, la interpretabilidad del *Random Forest* —mediante importancia de características y reglas de árbol legibles— representa una ventaja operacional significativa sobre los modelos de caja negra.

La elección de *Random Forest* sobre otros algoritmos clásicos para la clasificación de manifiestos IaC responde a tres criterios documentados en la literatura de aprendizaje automático aplicado a la seguridad:

1. **Robustez ante datos binarios y escasos.** Las características del presente trabajo son en su mayoría binarias (presencia/ausencia de una misconfiguration), con alta proporción de ceros. Los *Random Forests* manejan este tipo de datos de forma nativa sin requerir normalización, a diferencia de las Máquinas de Vectores Soporte (SVM) o las Redes Neuronales que son sensibles a la escala de las características (Breiman, 2001).

2. Gestión intrínseca del desequilibrio de clases. El parámetro

`class_weight=balanced` de *scikit-learn* implementa una estrategia de ponderación inversamente proporcional a la frecuencia de clase, lo que eleva efectivamente el coste de clasificar erróneamente las muestras de la clase minoritaria (misconfiguraciones críticas) sin modificar la función de pérdida subyacente (Pedregosa y cols., 2011).

3. Generación de probabilidades calibradas. El promedio de los votos del

ensemble de árboles produce probabilidades suaves en $[0, 1]$ que pueden utilizarse directamente como características continuas (*risk_scores*) en el grafo de la Capa 2. Algoritmos como los Árboles de Decisión individuales o los *k*-Nearest Neighbours producen probabilidades menos calibradas en presencia de desequilibrio de clases.

La Tabla 2.1 resume esta comparativa entre algoritmos candidatos para la Capa 1.

Tabla 2.1: Comparativa de algoritmos de ML para clasificación de manifiestos IaC

Algoritmo	Binario	Desequilibrio	Prob. cali- brada	Observación
Random Forest	✓	✓	Buena	Adoptado (Capa 1)
Árbol de Decisión	✓	Parcial	Pobre	Alta varianza en grafos
Support Vector Machine (Máqui- na de Vectores Soporte) (SVM) (RBF)	✓	Parcial	Requiere Platt	Entrenamiento $O(n^2)$ – $O(n^3)$; viable pero menos eficiente
Red Neuronal (MLP)	✓	Con pesos	Variable	Requiere normalización
<i>k</i> -NN	✓	No	Estimada	Costoso en inferencia

Fuente: elaboración propia.

Trabajos como Wist, Hammer, y Yazidi 2021, que aplican técnicas de Machine Learning (Aprendizaje Automático) (ML) a la predicción de vulnerabilidades en código fuente, confirman que los *Random Forests* ofrecen sistemáticamente mejor rendimiento que los Árboles de Decisión individuales y comparable al de las Redes Neuronales superficiales en espacios de características binarias de alta dimensión. Esta evidencia, combinada con las ventajas de interpretabilidad para auditorías de seguridad, motivó la adopción del *Random*

Forest como componente de Capa 1 de *kubescan*.

2.4. Redes neuronales de grafos

Las redes neuronales de grafos (Graph Neural Network (Red Neuronal de Grafos) (GNN)s) han emergido como la herramienta principal para aprendizaje en datos con estructura relacional (Hamilton, 2020; Zhou y cols., 2020). A diferencia de los clasificadores tabulares clásicos, las GNNs operan directamente sobre la topología del grafo, propagando información entre vecinos mediante operaciones de paso de mensajes (*message passing*). Formalmente, en una capa GNN la representación del nodo v se actualiza como:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathbf{W}^{(l)} \cdot \text{AGGREGATE}\left(\left\{\mathbf{h}_u^{(l)} : u \in \mathcal{N}(v)\right\}\right) + \mathbf{b}^{(l)}\right) \quad (2.1)$$

donde $\mathcal{N}(v)$ es el conjunto de vecinos de v , $\mathbf{W}^{(l)}$ y $\mathbf{b}^{(l)}$ son parámetros entrenables de la capa l , y σ es una función de activación no lineal. La elección de la función AGGREGATE diferencia las principales arquitecturas.

Las arquitecturas más relevantes para este trabajo son:

Graph Convolutional Networks (GCN) (Kipf y Welling, 2017): implementan AGGREGATE como la media ponderada de los estados de los vecinos normalizada por sus grados. Son eficientes pero asignan pesos uniformes a todos los vecinos, sin distinción entre tipos de arista.

GraphSAGE (Hamilton, Ying, y Leskovec, 2017): extensión inductiva que permite aplicar el modelo a grafos no vistos durante el entrenamiento, relevante en escenarios donde aparecen nuevos clústeres en producción. La inducción se logra aprendiendo funciones de agregación en lugar de *embeddings* fijos.

Graph Attention Networks (GAT) (Veličković y cols., 2018): sustituyen la normalización fija de Graph Convolutional Network (Red Convolutiva de Grafos) (GCN) por coeficientes de atención aprendidos. Para cada par de nodos (v, u) el coeficiente de atención es:

$$\alpha_{vu} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_u]))}{\sum_{k \in \mathcal{N}(v)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_k]))} \quad (2.2)$$

donde $\mathbf{a}$ es un vector de parámetros de atención y $\parallel$ denota concatenación. Este diseño es especialmente apropiado para grafos de clúster *Kubernetes*, donde una arista de privilegio (*privilege_reach*) debe pesar más que una arista de proximidad de directorio (*dir_proximity*). El modelo *How Powerful are GNNs?* de Xu, Hu, Leskovec, y Jegelka 2019 establece que la capacidad discriminativa de las GNNs está acotada por el test de isomorfismo de Weisfeiler-Lehman, y que la suma de vecinos maximiza el poder expresivo frente a la media o el máximo. En este trabajo se adopta un *pooling* combinado de media y máximo por su robustez ante grafos de tamaño muy variable, aceptando explícitamente ese compromiso de expresividad. Esta intuición motiva el uso de *pooling* combinado de media y máximo en la Capa 2.

2.5. GNNs aplicadas a ciberseguridad

La aplicación de GNNs a la detección de amenazas es un campo emergente con rápido crecimiento. Bilot, Madhoun, Agha, y Zouaoui 2023 proporcionan una revisión exhaustiva de los enfoques basados en GNN para la detección de intrusiones en red, identificando tres paradigmas: detección de anomalías en grafos de flujo, clasificación de nodos en grafos de comunicación, y clasificación de grafos de comportamiento por muestra. Una segunda revisión publicada en *Computers and Security* en 2024 (Zhong, Lin, Zhang, y Xu, 2024) amplía esta taxonomía y documenta un crecimiento sostenido de la investigación en el área en los últimos años.

E-GraphSAGE (Lo, Layeghy, Sarhan, Gallagher, y Portmann, 2022) propone representar flujos de red como grafos de transacción para clasificación de intrusiones en IoT, alcanzando tasas de detección superiores a los clasificadores tabulares clásicos en el dataset UNSW-NB15. La analogía con el presente trabajo es directa: los manifiestos *Kubernetes* son los nodos y las relaciones de privilegio y movimiento lateral son las aristas del grafo de amenaza. Una diferencia clave es que *E-GraphSAGE* modela flujos de tráfico en tiempo real, mientras que este trabajo modela configuraciones declarativas estáticas, lo que permite operar antes del despliegue.

Li, Zeng, Chen, y Liang 2022 construyen grafos de conocimiento de técnicas de ataque a partir de informes de inteligencia de amenazas (CTI) y demuestran que el razonamiento estructural sobre relaciones entre técnicas mejora la predicción de campañas de ataque. Este enfoque comparte con el presente trabajo la intuición central de que el grafo de rela-

ciones entre recursos es más informativo que la suma de sus nodos individuales. Aunque opera sobre conocimiento de ataques pasados (retrospectivo) y no sobre configuraciones declarativas (preventivo), la arquitectura basada en grafos de conocimiento inspiró el diseño de los tipos de arista semánticamente diferenciados del presente sistema.

Una limitación reconocida en la literatura es la escasez de conjuntos de datos de grafos etiquetados de alta calidad para ciberseguridad. Tanto Bilot y cols. 2023 como Zhong y cols. 2024 identifican la construcción de benchmarks estandarizados como el principal obstáculo para el avance del área, lo que refuerza la contribución del presente trabajo al generar un conjunto de datos original de 599 grafos de clúster (1.154 tras aumentación de datos) con etiquetas de cadena de ataque.

2.6. Detección de cadenas de ataque y grafos de ataque en Kubernetes

La noción de cadena de ataque (*kill chain*) fue formalizada originalmente por Hutchins y cols. 2011 para describir las fases secuenciales de una intrusión avanzada: reconocimiento, weaponización, entrega, explotación, instalación, comando y control, y acción. En el contexto de *Kubernetes*, esta cadena se concreta de forma más compacta: requiere como mínimo (1) un pod con capacidad de escape al anfitrión (*TRUE_HOST_PID*, *privileged: true*, *hostPath* montado, etc.) y (2) un mecanismo de movimiento lateral hacia otros recursos del clúster (token de cuenta de servicio con permisos excesivos, montaje de secretos en el manifiesto, etc.).

En los últimos años han surgido dos herramientas de código abierto que abordan la detección de rutas de ataque en *Kubernetes* mediante grafos: *KubeHound* (Datadog Security Labs, 2023) e *IceKube* (WithSecure Labs, 2023).

KubeHound (Datadog, 2023) construye un grafo de ataque del clúster a partir de la consulta directa al servidor Application Programming Interface (API) de *Kubernetes*, almacena el resultado en JanusGraph y permite explorar las rutas de ataque más cortas mediante el lenguaje de consulta Gremlin. El proyecto fue motivado por la misma intuición que subyace al presente trabajo, formulada por John Lambert y citada en su documentación: «*los defensores piensan en listas, los atacantes piensan en grafos*» (Datadog Security Labs, 2023). *KubeHound* cubre un amplio conjunto de ataques y ha sido adoptado por equipos de seguridad en producción.

IceKube (WithSecure Labs, 2023) sigue un enfoque análogo, basado en Neo4j, y permite definir rutas de ataque mediante consultas Cypher. Modela un amplio conjunto de técnicas de ataque como relaciones en el grafo y puede encontrar rutas desde cualquier identidad de bajo privilegio hasta *cluster-admin*.

La diferencia fundamental entre estas herramientas y *kubescan* es de paradigma: KubeHound e IceKube operan sobre un **clúster activo**, requiriendo acceso con privilegios elevados a `kubectl`. *kubescan*, en cambio, opera sobre **manifiestos YAML estáticos** antes del despliegue, integrándose en el pipeline CI/CD como un control de seguridad *shift-left*. Además, ambas herramientas utilizan búsqueda de rutas mediante consultas ad-hoc en bases de datos de grafos, sin componente de aprendizaje automático supervisado; *kubescan* es el primer sistema, según la revisión bibliográfica realizada en este trabajo (Sección 2.7), que combina análisis estático de manifiestos con GNNs para la predicción de cadenas de ataque como tarea de clasificación supervisada.

2.7. Síntesis y posicionamiento

El análisis de la literatura permite identificar tres carencias fundamentales que motivan el presente trabajo. En primer lugar, las herramientas de análisis estático existentes (Checkov, kube-linter, Kubescape) operan a nivel de manifiesto individual y no modelan las relaciones estructurales entre recursos del clúster que habilitan las cadenas de ataque. En segundo lugar, los conjuntos de datos de referencia para seguridad de *Kubernetes* son escasos, de pequeño tamaño y desequilibrados hacia las clases de riesgo bajo, lo que dificulta el entrenamiento de clasificadores de clúster: el propio análisis del dataset de Rahman et al. confirma que `TRUE_HOST_PID` —la flag de escape más peligrosa— aparece en sólo el 0,3 % de los manifiestos reales. En tercer lugar, las herramientas de grafo de ataque más avanzadas (KubeHound, IceKube) requieren un clúster activo, lo que impide su uso como control preventivo en el pipeline CI/CD.

La Tabla 2.2 resume el posicionamiento de *kubescan* frente a las principales herramientas del estado del arte.

El sistema *kubescan* presentado en este trabajo responde a las tres carencias identificadas: adopta una arquitectura de grafo heterogéneo con tipos de arista semánticamente significativos para el dominio *Kubernetes*, combina el análisis estático (Capa 1) con el razonamiento estructural (Capa 2) en un *ensemble* optimizado (Capa 3), y contribuye un

Tabla 2.2: Comparativa de herramientas de seguridad para Kubernetes

Herramienta	Análisis estático	Requiere clúster	Grafos	ML supervisado
Checkov (Bridgecrew, 2021)	✓	No	No	No
kube-linter (StackRox, 2021)	✓	No	No	No
Kubescape (ARMO Ltd., 2021)	✓	Opcional	No	No
KubeHound (Datadog Security Labs, 2023)	No	Sí	✓	No
IceKube (WithSecure Labs, 2023)	No	Sí	✓	No
kubescan	✓	No	✓	✓

Fuente: elaboración propia.

conjunto de datos de 599 grafos de clúster originales con etiquetas de cadena de ataque derivadas de reglas de seguridad explícitas y auditables. Al operar sobre manifiestos YAML sin requerir acceso al clúster, *kubescan* se posiciona como un control preventivo integrable en pipelines CI/CD, complementario a —y no sustituto de— las herramientas de análisis de clúster activo como KubeHound.

3. Objetivos y Metodología

Este capítulo define los objetivos del trabajo y describe la metodología seguida para alcanzarlos. En primer lugar se formula el objetivo general y los objetivos específicos; los de naturaleza cuantificable incorporan criterios de éxito medibles. A continuación se describen las cinco fases del proceso de desarrollo, la estrategia de partición y validación de datos, y la justificación de las métricas de evaluación adoptadas.

3.1. Objetivo general

Diseñar, implementar y evaluar un sistema automático de predicción de cadenas de ataque en infraestructuras *Kubernetes* que complemente los enfoques de análisis estático por manifiesto individual con el contexto estructural del clúster, mediante la combinación de un clasificador de manifiestos individuales (*Random Forest*) con una red neuronal de grafos (*Graph Attention Network*) y optimización de ensemble mediante Algoritmo Genético. El sistema opera principalmente sobre manifiestos YAML estáticos, sin requerir acceso a un clúster activo, si bien incorpora opcionalmente una vía de consulta directa al clúster (OE6).

3.2. Objetivos específicos

- OE1.** Construir un conjunto de datos de manifiestos *Kubernetes* etiquetados que combine múltiples fuentes públicas, contenga al menos 2.000 manifiestos, y alcance una cobertura mínima del 2 % en la flag de escape `TRUE_HOST_PID`, superando el 0,3 % de los repositorios reales no aumentados.
- OE2.** Desarrollar una Capa 1 de clasificación de manifiestos individuales basada en *Random Forest* con F1 macro superior a 0,85, que produzca *risk scores* calibrados para usar como características de nodo en la Capa 2.
- OE3.** Diseñar un esquema de grafo de clúster con tipos de arista semánticamente significativos para el dominio *Kubernetes*: proximidad de directorio, alcance de privilegio, movimiento lateral mediante cuenta de servicio, co-namespacio semántico y escalada RBAC; su contribución se valida mediante ablación frente a una GCN sin tipos de arista bajo idéntico protocolo de evaluación.

- OE4.** Desarrollar una Capa 2 de clasificación de clústeres basada en GAT que alcance Precisión@5 superior a 0,70 (media de validación cruzada de 5 pliegues) en la detección de cadenas de ataque.
- OE5.** Implementar una Capa 3 que optimice mediante Algoritmo Genético una función objetivo combinada de Precisión@5 y falsos positivos de clústeres limpios sobre las predicciones *out-of-fold* de la validación cruzada, con criterios de éxito sobre el conjunto de test reservado: False Positive Rate (Tasa de Falsos Positivos) (FPR)_{clean} = 0 y Precisión@5 no inferior a la del mejor componente individual del *ensemble*.
- OE6.** Empaquetar el sistema completo como una herramienta de línea de comandos (*kubescan*) con soporte para análisis de directorios locales y, opcionalmente, consulta directa al API de *Kubernetes* mediante *kubectrl*; instalable en entornos Python 3.10+ mediante un único `pip install`.

3.3. Metodología

El trabajo sigue un proceso iterativo de cinco fases que combina el paradigma de desarrollo de *software* (requisitos, diseño, implementación, prueba) con el proceso de ciencia de datos (adquisición, preparación, modelado, evaluación):

Fase 1 — Adquisición de datos. Descarga y normalización de manifiestos *Kubernetes* de cinco fuentes públicas complementarias (agrupadas en cuatro categorías): el conjunto de datos de Rahman et al. (Rahman y cols., 2023) (repositorios reales etiquetados), BishopFox BadPods (Art, 2021) (pods intencionadamente vulnerables), *Kubernetes Goat* (Akula, 2020) (escenarios de vulnerabilidad) y repositorios de seguridad adicionales (CTFs, laboratorios y *fixtures*). El criterio de inclusión de repositorios adicionales fue la presencia de al menos una flag de escape activa (`HOSTPATH_MOUNT`, `TRUE_HOST_PID`, etc.) en al menos un manifiesto del directorio.

Fase 2 — Extracción de características y construcción del grafo. Extracción de 28 características por manifiesto (25 indicadores binarios y 3 agregados derivados) mediante análisis estático multi-herramienta (Sección 4.9) y construcción del grafo de clúster con cinco tipos de arista (Tabla 4.2). En esta fase se realizó también la validación de cobertura de la flag `HOSTPATH_MOUNT`, documentada en la Sección 5.2.

Fase 3 — Entrenamiento de la Capa 1. Entrenamiento de un clasificador *Random Forest* mediante validación cruzada de 5 pliegues para predecir la presencia de misconfiguraciones individuales y generar *risk scores* calibrados, con los hiperparámetros fijos detallados en la Sección 4.5.

Fase 4 — Entrenamiento de la Capa 2. Entrenamiento de una GAT de 3 capas con agrupamiento de grafos (*graph pooling*) y validación cruzada estratificada de 5 pliegues consciente de grupos y de familias de plantilla: las variantes aumentadas siguen a su grafo base y las familias de plantilla con etiquetas mixtas (p. ej. `badpods_*`) se asignan como unidades atómicas a una única partición, nunca repartidas entre pliegues. La aumentación de datos se aplica sobre los grafos de entrenamiento para mitigar la escasez de ejemplos de cadena de ataque, y el *checkpoint* de cada pliegue se selecciona por Precisión@5 de validación.

Fase 5 — Optimización de ensemble y evaluación. Optimización de los pesos del ensemble de tres componentes (RF, GNN, señal de escape) mediante Algoritmo Genético sobre las predicciones *out-of-fold* de la validación cruzada de la Capa 2, aplicación de una restricción de viabilidad estructural en el *ranking* final (un grafo de un solo nodo no puede albergar una cadena de ataque multi-salto), y evaluación final sobre el conjunto de test reservado desde el inicio y no utilizado en ninguna fase de entrenamiento ni selección de hiperparámetros.

3.4. Estrategia de partición y validación de datos

La estrategia de partición de datos difiere entre la Capa 1 (datos tabulares) y la Capa 2 (datos de grafos), dado que sus tamaños y distribuciones son distintos. El tamaño final de cada conjunto depende del proceso de adquisición y construcción descrito en las Secciones 4.3 y 4.4 del Capítulo 4.

Para la Capa 1, el conjunto de manifiestos se divide en un conjunto de entrenamiento (80 %) y un conjunto de test (20 %), con estratificación por la variable objetivo para mantener la proporción de clases en ambos subconjuntos. La evaluación del modelo se realiza mediante validación cruzada de 5 pliegues sobre el conjunto de entrenamiento para la selección de hiperparámetros, y el conjunto de test se reserva exclusivamente para la evaluación final.

Para la Capa 2 se reserva desde el inicio una partición de test de clústeres originales, y el

resto se evalúa mediante validación cruzada estratificada de 5 pliegues, con estratificación por etiqueta de clúster (*clean*, *isolated*, *attack_chain*) y particionado consciente de grupos y de familias de plantilla (Capítulo 4). En cada pliegue, la aumentación de datos se aplica **exclusivamente** sobre los grafos de entrenamiento del pliegue, nunca sobre los de validación, evitando así la fuga de datos. La validación cruzada complementa al conjunto de test reservado porque es la estrategia estándar para conjuntos de grafos pequeños (Hamilton, 2020): con pocas muestras de la clase minoritaria, una única partición de validación no sería representativa por sí sola.

3.5. Métricas de evaluación

La selección de métricas de evaluación responde a los requisitos operacionales del sistema y a las características del dataset.

F1 macro para la Capa 1 y la clasificación por clases de la Capa 2: la F1 macro calcula la media no ponderada de la F1 de cada clase, otorgando igual importancia a las clases minoritarias (misconfiguraciones críticas) que a las mayoritarias (manifiestos seguros). Esta propiedad es fundamental en datasets de seguridad desequilibrados, donde la exactitud (*accuracy*) sería una métrica engañosa —sobre el conjunto de test de la Capa 1 (566 muestras), un clasificador que predice siempre «seguro» obtendría una exactitud del 56,3 % pero F1 macro del 36,0 %.

AUC-ROC para el modelo binario: mide la capacidad del clasificador para separar las dos clases independientemente del umbral de decisión, lo que permite al operador ajustar la sensibilidad según las necesidades del entorno.

Precisión@k ($P@k$) para la Capa 2: en lugar de evaluar la clasificación por umbral, $P@k$ mide la fracción de clústeres de cadena de ataque en las primeras k posiciones del ranking por puntuación de riesgo. Esta métrica está directamente alineada con el caso de uso operacional: un equipo de seguridad que audita los k clústeres de mayor riesgo en su infraestructura quiere que todos sean cadenas de ataque reales, no falsas alarmas. La elección de $k = 5$ refleja la capacidad de revisión manual típica de un equipo de seguridad reducido en una jornada laboral.

FPR_clean (tasa de falsos positivos sobre clústeres limpios): mide la fracción de clústeres limpios entre los k primeros del *ranking* de riesgo¹. Un $FPR_clean = 0$ significa que

¹Operacionalmente es una tasa de limpios en el top- k , no una tasa de falsos positivos en sentido estricto

ningún clúster limpio aparece entre los de mayor puntuación, eliminando las investigaciones innecesarias.

(que normalizaría por el total de clústeres limpios); se mantiene la denominación `FPR_clean` por coherencia con la implementación.

4. Diseño e Implementación

Este capítulo describe el diseño completo del sistema *kubescan* y su implementación como herramienta de software distribuible. Se presentan, en primer lugar, los requisitos funcionales y no funcionales que guiaron las decisiones de diseño; a continuación se detalla la arquitectura de tres capas del sistema, los procesos de construcción del conjunto de datos y del grafo de clúster, y los algoritmos de entrenamiento de cada capa; finalmente se describe la interfaz de línea de comandos resultante. La implementación completa se encuentra disponible en el repositorio indicado en el Apéndice A.

4.1. Identificación de requisitos

El diseño de *kubescan* parte de un análisis de requisitos derivado tanto de las limitaciones identificadas en el estado del arte (Sección 2.7) como de los vectores de ataque documentados por la NSA y la Cybersecurity and Infrastructure Security Agency (CISA) (National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022).

4.1.1. Requisitos funcionales

RF-1 — Detección de misconfiguraciones individuales. El sistema debe clasificar cada manifiesto YAML como seguro o *misconfigured*, produciendo además un índice de severidad en tres niveles (limpio, bajo–medio, alto–crítico).

RF-2 — Detección de cadenas de ataque. El sistema debe clasificar un conjunto de manifiestos que describe un clúster en una de tres categorías: *clean*, *isolated* (misconfiguraciones sin cadena viable) o *attack_chain* (ruta de escalada de privilegios hasta movimiento lateral).

RF-3 — Interfaz de línea de comandos. El sistema debe exponer un comando `kubescan scan <dir>` para análisis de directorios locales y un comando `kubescan live` para análisis de clústeres activos mediante *kubectl*.

RF-4 — Formatos de salida. El sistema debe producir informes en formato texto legible y en formato JSON estructurado para integración con herramientas externas.

RF-5 — Priorización de riesgos. El sistema debe producir un ranking de clústeres

por puntuación de riesgo que maximice la Precisión@5 — colocando las cadenas de ataque reales en las primeras posiciones.

4.1.2. Requisitos no funcionales

RNF-1 — Trazabilidad. Todas las reglas de detección de nodos de escape y movimiento lateral deben ser explícitas, auditables y derivadas de fuentes normativas documentadas (Art, 2021; National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022).

RNF-2 — Reproducibilidad. Los modelos entrenados deben almacenarse como *checkpoints* e incluirse en el paquete, de forma que el análisis sea idempotente dado el mismo directorio de entrada.

RNF-3 — Instalabilidad. El sistema debe ser instalable mediante `pip install -e kubescan/` sin dependencias de compiladores externos.

RNF-4 — Latencia. El análisis de un clúster de hasta 500 manifiestos debe completarse en menos de 60 segundos en hardware de gama media (CPU).

4.2. Arquitectura general del sistema

El sistema *kubescan* implementa una arquitectura de tres capas que transforma un directorio de manifiestos YAML en una puntuación de riesgo de cadena de ataque (Figura 4.1):

1. **Capa 1 (Random Forest):** extrae 28 características por manifiesto (25 indicadores binarios y 3 agregados derivados) y produce un *risk_score* $\in [0, 1]$.
2. **Capa 2 (GAT):** modela el clúster como un grafo donde los nodos son manifiestos y las aristas representan relaciones de seguridad; produce una probabilidad de cadena de ataque $\in [0, 1]$.
3. **Capa 3 (Ensemble GA):** combina las señales de las dos capas anteriores con una señal binaria de escape según la ecuación:

$$\text{score}(C) = w_{\text{rf}} \cdot \overline{\text{risk}}(C) + w_{\text{gnn}} \cdot p_{\text{chain}}(C) + w_{\text{esc}} \cdot \mathbf{1}[\exists \text{ nodo de escape en } C] \quad (4.1)$$

donde los pesos w_{rf} , w_{gnn} , w_{esc} son optimizados mediante Algoritmo Genético sobre las predicciones *out-of-fold* de la validación cruzada (Sección 4.7).

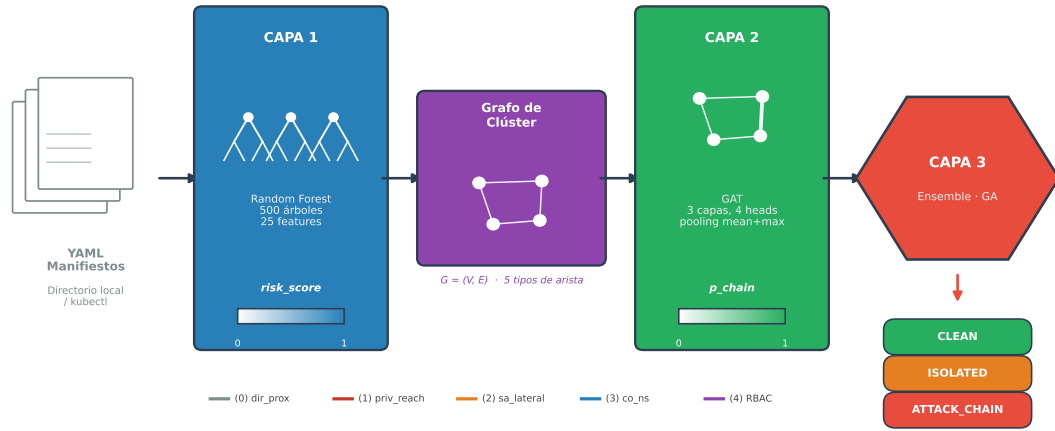


Figura 4.1: Arquitectura del sistema *kubescan*: pipeline de tres capas desde manifiestos YAML de entrada hasta la clasificación de riesgo del clúster. Los *risk_scores* producidos por la Capa 1 se incorporan como atributo nodal 26 del grafo de entrada de la Capa 2.

Fuente: elaboración propia.

4.3. Construcción del dataset

4.3.1. Fuentes de datos

El conjunto de datos tabular final contiene 2.827 filas y 58 columnas, combinando cinco fuentes (agrupadas en cuatro categorías, dado que Rahman et al. aporta dos plataformas distintas con características de distribución diferentes; ver Tabla 4.1):

La incorporación de BadPods fue esencial para mitigar la escasez de ejemplos de misconfiguraciones críticas en los repositorios reales: antes de añadir esta fuente, `TRUE_HOST_PID` aparecía en sólo el 0,3 % de las filas; tras la incorporación, la cobertura ascendió al 3,8 %. En cuanto a los conjuntos de datos de referencia existentes para intrusión en red, como UNSW-NB15 (Moustafa y Slay, 2015), se estudió su metodología de construcción como guía para el diseño del proceso de etiquetado y la estrategia de balanceo de clases, dado que presentan un desequilibrio de clases del mismo orden de

Tabla 4.1: Composición del conjunto de datos de manifiestos Kubernetes

Fuente	Filas	Rol
Rahman et al. (Rahman y cols., 2023) — GitHub	1.510	Manifiestos reales, etiquetados
Rahman et al. (Rahman y cols., 2023) — GitLab	396	Ídem, plataforma distinta
BishopFox BadPods (Art, 2021)	128	Pods vulnerables intencionales
Kubernetes Goat (Akula, 2020)	14	Escenarios de vulnerabilidad
Repositorios de seguridad adicionales	779	CTFs, <i>fixtures</i> de herramientas, labs
Total	2.827	

Fuente: elaboración propia.

magnitud que el de este trabajo (56,3 % seguros vs. 43,7 % misconfigured). El conjunto *GenKubeSec* (Malul y cols., 2024), publicado en 2024, no estaba disponible durante el desarrollo de este trabajo; en su lugar se empleó *Checkov* (Bridgecrew, 2021) directamente sobre los ficheros YAML descargados como fuente de validación multi-herramienta equivalente.

4.3.2. Características extraídas

El extractor de características aplica 28 reglas de análisis estático a cada manifiesto YAML, organizadas en tres grupos:

- **18 flags Rahman** (subconjunto del trabajo original): `TRUE_HOST_PID`, `CAP_SYS_ADMIN`, `SEC_CONT_OVER_PRIVIL`, `INSECURE_HTTP`, etc.
- **3 features derivadas**: `cap_misuse` (OR de las dos flags de capacidades de sistema), `all_secrets` (OR de los indicadores de exposición de secretos), `total_misconfigs` (suma de todas las flags activas).
- **7 features extendidas**: `NO_RUN_AS_NON_ROOT`, `IMAGE_USES_LATEST`, `SA_AUTOMOUNT_TOKEN`, `HOSTPATH_MOUNT`, etc.; imputadas mediante *Checkov* para el 47,4 % de filas con YAML descargado, y mediante la mediana de columna para el resto.

La Capa 2 (GNN) consume únicamente las 18 flags Rahman y las 7 features exten-

didadas (25 en total, `FEATURE_COLS` en `yaml_parser.py`) como vector de nodo, excluyendo las 3 features derivadas —agregados calculados a partir de las otras 25 que no aportan información estructural adicional al grafo—; ver Sección 4.9.

Tres features presentan varianza casi nula pero se mantienen en el entrenamiento (con importancia residual o nula, Sección 5.1.2): `SECCOMP_UNCONFINED` (15/2.827 activa), `VALID_TAINT_SECRET` (0/2.827 activa) y `NO_NETWORK_POLICY` (2.818/2.827 activa, sin poder discriminativo).

4.4. Construcción del grafo de clúster

Cada directorio de manifiestos se representa, mediante *NetworkX* (Hagberg, Schult, y Swart, 2008), como un grafo dirigido $G = (V, E)$ donde $|V|$ es el número de manifiestos del clúster y E contiene aristas de cinco tipos semánticamente distintos (Tabla 4.2). La Figura 4.2 ilustra un ejemplo con cuatro nodos que incluye los tres tipos de nodo posibles: escape, movimiento lateral y limpio.

Tabla 4.2: Tipos de aristas en el grafo de clúster

Tipo	Código	Dirección	Criterio
Proximidad de directorio	0	Bidireccional	Mismo subdirectorio YAML
Alcance de privilegio	1	Dirigida (escape→todos)	Flag de escape activa
Movimiento lateral SA	2	Dirigida (lateral→todos)	Flag de movimiento lateral
Co-espacio de nombres (<code>SEMANTIC_NS</code>)	3	Bidireccional	Mismo <i>namespace</i> K8s
Escalada RBAC	4	Dirigida (elevado→todos)	Service Account (Cuenta de Servicio de Kubernetes) (SA) vinculado a rol privilegiado

Fuente: elaboración propia.

Cada nodo v_i lleva un vector de características $\mathbf{x}_i \in \mathbb{R}^{26}$: índices 0–24 corresponden a las 25 features binarias de la Capa 1 y el índice 25 contiene el *risk_score* producido por el modelo *Random Forest*.

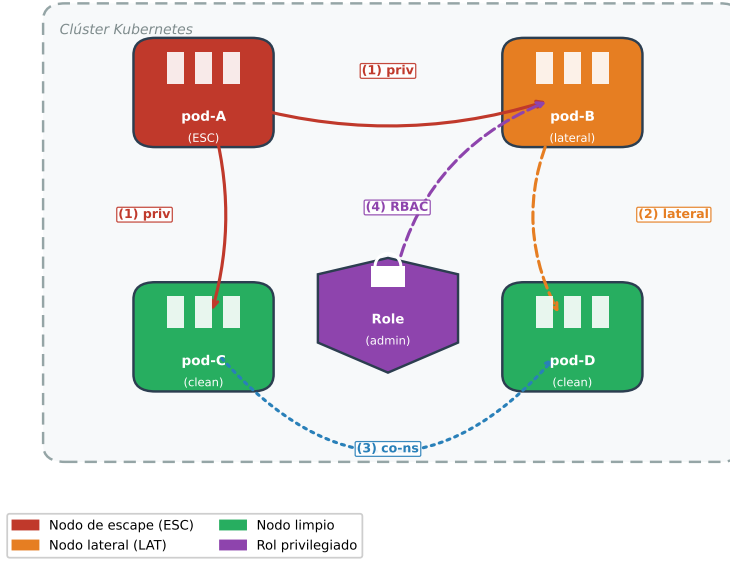


Figura 4.2: Ejemplo de grafo de clúster Kubernetes con cuatro nodos, mostrando los tres tipos de nodo (escape, movimiento lateral y limpio) y las aristas dirigidas que cada uno origina.

Fuente: elaboración propia.

Como se observa en la Figura 4.2, el nodo rojo (pod-A) tiene flags de escape activas y emite aristas de tipo 1 (*privilege_reach*) hacia todos los demás nodos del clúster. El nodo naranja (pod-B) tiene flags de movimiento lateral y emite aristas de tipo 2 (*sa_lateral*). La arista de tipo 4 (RBAC) representa la vinculación de un *ServiceAccount* a un rol privilegiado.

El dataset de grafos final comprende 599 grafos originales (25 clean, 537 isolated, 37 attack_chain) y 1.154 grafos totales tras la aumentación de datos aplicada exclusivamente en el conjunto de entrenamiento. Esta cifra incorpora el corpus completo de Rahman et al. (Rahman y cols., 2023), cuyos *namespaces* cruzados se resolvieron mediante descarga autenticada desde GitHub, lo que amplió sustancialmente el número de clústeres *isolated* respecto a las fases experimentales preliminares descritas en la Sección 5.2 del Capítulo 5.

4.4.1. Detección de escape y movimiento lateral

La clasificación de nodos utiliza dos conjuntos de flags definidos en el código como constantes inmutables (**frozenset**). Un nodo se clasifica como de **escape** si tiene activa al menos una de las siguientes 8 flags: **TRUE_HOST_PID**, **TRUE_HOST_IPC**, **TRUE_HOST_NET**,

`DOCKERSOCK_PATH`, `CAP_SYS_ADMIN`, `CAP_SYS_MODULE`, `SEC_CONT_OVER_PRIVIL` o `HOSTPATH_MOUNT`. Este último vector fue incorporado durante el desarrollo al detectar que *badpods_hostpath* —que monta el sistema de ficheros del anfitrión— era incorrectamente etiquetado como *isolated* al no tener ninguna otra flag de escape activa. Con la corrección, el número de clústeres de cadena de ataque del corpus preliminar de 471 grafos ascendió de 13 a 15 (37 en el corpus final de 599).

Un nodo se clasifica como de **movimiento lateral** si tiene activa al menos una de las siguientes 4 flags: `SA_AUTOMOUNT_TOKEN` (token de cuenta de servicio automontado), `USES_DEFAULT_SA` (usa la cuenta de servicio por defecto del *namespace*), `WITHIN_MANIFEST_SECRET` (secreto referenciado en el manifiesto) o `ALLOW_PRIVI` (contenedor con capacidad de escalada). Las aristas de movimiento lateral (tipo 2) se construyen de forma análoga a las de escape: se emite una arista dirigida desde cada nodo lateral hacia todos los demás, pero únicamente si no existe ya una arista de mayor prioridad (escape o RBAC).

4.4.2. Detección de privilegio RBAC

La construcción de aristas RBAC (tipo 4) requiere dos pasadas sobre el conjunto de manifiestos. La primera pasada identifica los roles privilegiados: aquellos cuyo nombre está en el conjunto `{cluster-admin, admin, edit}` o cuyas reglas contienen verbos de escalada `{*, escalate, bind, impersonate}`. La segunda pasada recorre los objetos *RoleBinding* y *ClusterRoleBinding*: sólo cuando el campo *roleRef* referencia un rol privilegiado identificado en la primera pasada se emite una arista RBAC desde el pod que usa esa cuenta de servicio hacia todos los demás nodos del clúster.

4.5. Capa 1: Random Forest

4.5.1. Modelo binario

Se entrena un *RandomForestClassifier* de *scikit-learn* (Pedregosa y cols., 2011) con los hiperparámetros siguientes:

- `n_estimators = 500`
- `max_features = sqrt` (clasificación estándar)
- `min_samples_leaf = 2` (evita hojas de muestra única)

- `class_weight = balanced` (compensa ratio 1,29:1)
- `random_state = 42`

El modelo se entrena sobre 2.261 muestras (80 % del total) y se evalúa en 566 muestras de test no vistas durante el entrenamiento. La probabilidad predicha para la clase *misconfigured* (clase 1) se usa como *risk_score* del nodo en el grafo.

4.5.2. Modelo de severidad

Además del modelo binario, se entrena un clasificador de severidad de tres clases (*clean*, *low-medium*, *high-critical*) para ofrecer información más granular al operador de seguridad. Este modelo no interviene en el cálculo del grafo; se incluye como salida adicional de la Capa 1.

4.6. Capa 2: Graph Attention Network

4.6.1. Arquitectura

El modelo GAT (Veličković y cols., 2018) implementado tiene la siguiente estructura, con las dimensiones exactas verificadas en `gat_encoder.py`:

1. **Embedding de tipo de arista:** `Embedding(5, 8)`. Los cinco tipos de arista (proximidad de directorio, alcance de privilegio, lateral por *ServiceAccount*, espacio de nombres y RBAC) son categóricos: codificarlos como un escalar continuo implicaría un orden espurio (el tipo 4 «valdría» cuatro veces el tipo 1). Cada tipo se proyecta a un vector aprendido de 8 dims que `GATConv` consume como atributo de arista (*edge_dim=8*).
2. **Proyección inicial:** `Linear(26, 256) + LayerNorm(256) + ReLU + Dropout(0.3)`. Proyecta el vector de nodo de 26 dims a 256 dims (64×4 cabezas).
3. **Capas GAT 0 y 1** (*concat=True*): `GATConv(256, 64, heads=4)` con *edge_dim=8*; salida 256 dims (64×4); seguidas de `LayerNorm`, ELU, conexión residual y `Dropout(0.3)`.
4. **Capa GAT 2** (*concat=False, heads=1*): `GATConv(256, 64, heads=1)`; salida 64 dims (una sola cabeza de atención que produce directamente el embedding final)

del nodo, sin concatenación); seguida de `LayerNorm`, `Exponential Linear Unit (ELU)` y `Dropout(0.3)` (sin conexión residual: las dimensiones de entrada y salida difieren, 256 frente a 64).

5. **Global pooling:** `global_mean_pool` y `global_max_pool` sobre los 64-dim embeddings de todos los nodos; concatenación $\rightarrow$ 128 dims (64 + 64).

6. **Clasificador Multi-Layer Perceptron (Perceptrón Multicapa) (MLP):**

`Linear(128, 64) + Rectified Linear Unit (ReLU) + Dropout(0.3) + Linear(64, 3)`.

La elección de GAT sobre GCN responde a que el mecanismo de atención permite al modelo aprender qué aristas son más informativas para la señal de ataque: una arista de *privilege_reach* debe recibir mayor peso que una arista de proximidad de directorio. El *pooling* combinado de media y máximo captura dos señales complementarias: la media refleja la postura de seguridad promedio del clúster, mientras que el máximo identifica el nodo más peligroso, ambas necesarias para detectar cadenas de ataque que pueden estar concentradas en un único nodo crítico.

4.6.2. Entrenamiento

El modelo se entrena con *PyTorch Geometric* (Fey y Lenssen, 2019), construido sobre *PyTorch* (Paszke y cols., 2019), bajo los siguientes hiperparámetros: optimizador AdamW con tasa de aprendizaje 5×10^{-4} y *weight_decay* = 10^{-4} ; *scheduler* CosineAnnealingLR con `T_max`=300 y $\eta_{\text{mín}} = 10^{-5}$; función de pérdida CrossEntropyLoss con pesos de clase calculados por frecuencia inversa; *gradient clipping* con norma máxima 2,0; *early stopping* con paciencia de 50 épocas; tamaño de lote 8; semilla global 42 (idéntica para el entrenamiento y para la generación de particiones); y validación cruzada estratificada de 5 pliegues. El *checkpoint* de cada pliegue se selecciona por la mejor Precisión@5 de validación, con la F1 macro como criterio de desempate (`--select-by p5`), alineando la selección de modelos con la métrica primaria del sistema en lugar de con la métrica de clasificación.

Los pesos de clase se calculan en cada pliegue de entrenamiento mediante la siguiente fórmula implementada en `train_gnn.py`:

$$w_i = \frac{1/n_i}{\sum_{j=0}^{K-1} 1/n_j} \cdot K \quad (4.2)$$

donde n_i es el número de grafos de entrenamiento de la clase i y $K = 3$ es el número de clases. Esta fórmula normaliza los pesos para que su media sea 1,0, equivalente al comportamiento de `class_weight=balanced` de *scikit-learn*. Un detalle importante es que los pesos se calculan **después** de la aumentación de datos: como la aumentación genera 15 variantes por grafo de cadena de ataque, la clase *attack_chain* pasa de ser la minoría a ser la mayoría en el conjunto de entrenamiento (aproximadamente 380 grafos aumentados vs. 18 *clean* y 367 *isolated* en el conjunto de entrenamiento de cada pliegue, tras excluir las variantes de los clústeres de validación y test). La Ecuación 4.2 se auto-adapta a esta nueva distribución asignando mayor peso a *clean* e *isolated*, y menor peso a *attack_chain*, compensando el efecto de la sobre-representación creada por la aumentación.

4.6.3. Aumentación de datos

Para mitigar la escasez de ejemplos de cadena de ataque (37 de 599 grafos originales), se aplican tres estrategias de aumentación exclusivamente sobre los grafos de entrenamiento: *edge dropout* (eliminación aleatoria de entre el 10 % y el 35 % de aristas de los tipos 0 y 3, los menos críticos para la señal de ataque, con tasa muestreada uniformemente por variante); *feature masking* (puesta a cero, con probabilidad por nodo entre el 10 % y el 25 %, restringida a seis características de baja importancia — nunca las flags de escape, `HOSTPATH_MOUNT` ni el *risk_score*); y *subgraph sampling* (extracción de un subgrafo inducido por entre el 40 % y el 70 % de nodos, aplicada solo a grafos de más de 50 nodos, con *BFS* sembrada en el nodo de mayor *risk_score*; cada subgrafo se valida contra la regla de etiquetado de cadena y, si deja de cumplirla, se descarta en favor de una variante sobre el grafo completo, evitando introducir ruido de etiqueta). Cada grafo de cadena de ataque origina 15 variantes aumentadas, resultando en 555 grafos adicionales (37 originales $\times$ 15 variantes) en el *pool* de aumentación; de éstos, únicamente 390 (los correspondientes a los 26 grafos de cadena cuya base pertenece a la partición de entrenamiento) entran efectivamente en el entrenamiento, para un total de 1.154 grafos con aumentación.

La asignación de variantes a particiones es **consciente de grupos** (*group-aware*): una variante aumentada solo puede entrar en una partición de entrenamiento si su grafo base

pertenece a esa misma partición. Las variantes derivadas de clústeres de validación o de test se excluyen del entrenamiento por completo, ya que entrenar con casi-duplicados de un grafo de evaluación constituiría fuga de datos.

El particionado es además **consciente de familias de plantilla**: los clústeres originales que comparten una misma plantilla base (por ejemplo, las siete variantes `badpods_*`, que difieren únicamente en qué flag de escape activan, o las nueve `datadog_*`) son casi duplicados entre sí, no muestras independientes. Cuando una familia contiene etiquetas mixtas, sus miembros se asignan como una unidad atómica a una única partición — nunca se reparten entre entrenamiento, validación y test ni entre pliegues — y la asignación emplea un reparto voraz ponderado por el número de grafos de cada etiqueta, de modo que la proporción real de cada clase por partición siga las fracciones solicitadas. Sin esta regla, un modelo entrenado con cinco hermanos de una familia resuelve trivialmente al sexto en test, inflando las métricas. La partición resultante (semilla 42) es de 815 grafos de entrenamiento (con aumentación), 88 de validación y 86 de test. Adicionalmente, los 86 clústeres de test se mantienen fuera de los pliegues de validación cruzada: los pliegues particionan únicamente los 513 grafos originales restantes (32 de cadena), de modo que ni los modelos de pliegue ni las predicciones *out-of-fold* empleadas para ajustar los pesos del *ensemble* (Capa 3) observan ningún clúster de test, directa ni indirectamente.

4.7. Capa 3: Ensemble con Algoritmo Genético

Los pesos ($w_{\text{rf}}, w_{\text{gnn}}, w_{\text{esc}}$) de la Ecuación 4.1 son optimizados mediante un Algoritmo Genético (Holland, 1975) con 60 individuos codificados como pares $(w_{\text{gnn}}, w_{\text{esc}}) \in [0, 1]^2$, con w_{rf} derivado como $1 - w_{\text{gnn}} - w_{\text{esc}}$ (la representación 2D garantiza que la suma de los tres pesos sea siempre 1). La función objetivo es:

$$F = 0,7 \cdot P@5 + 0,3 \cdot (1 - \text{FPR}_{\text{clean}}) - \lambda \sum_i \text{máx}(0, w_{\text{mín}} - w_i) \quad (4.3)$$

El último término es una penalización de suelo con $w_{\text{mín}} = 0,1$ y $\lambda = 2,0$: sin ella, el optimizador es libre de anular cualquiera de los tres pesos cuando eso puntúa marginalmente mejor sobre la muestra *out-of-fold*, lo que produce soluciones degeneradas que no generalizan al conjunto de test. La penalización hace que ignorar una señal por completo esté estrictamente dominado salvo que la ganancia observada supere λ veces el déficit.

La selección se realiza mediante torneo de tamaño 3; el cruce es aritmético (media de los dos padres); la mutación aplica perturbación gaussiana con $\sigma = 0,08$ y tasa de mutación del 15 %, seguida de re-proyección al dominio válido ($w_{rf} \geq 0$); y el criterio de parada es 150 generaciones. Como *baseline* determinístico, se ejecuta previamente una búsqueda en rejilla 2D con paso 1/20 (231 evaluaciones); la comparación de su óptimo con el resultado del Genetic Algorithm (Algoritmo Genético) (GA) se discute en la Sección 5.3.

La señal de escape se define como un indicador binario que vale 1,0 si al menos un nodo del clúster tiene alguna flag de escape activa, y 0,0 en caso contrario. Esta formulación evita la dilución que produciría una fracción cuando la mayoría de los nodos del clúster son limpios.

La asignación de clase final a partir de la puntuación *ensemble_score* se realiza mediante los siguientes umbrales fijos, definidos en `ga_ensemble.py`:

- *ensemble_score* $\geq 0,60 \Rightarrow$ ATTACK_CHAIN
- *ensemble_score* $\geq 0,30 \Rightarrow$ ISOLATED_MISCONFIG
- De lo contrario $\Rightarrow$ CLEAN

Estos umbrales son fijos y no se reoptimizan durante el entrenamiento del GA, que optimiza únicamente los pesos w_i de la Ecuación 4.1. Permiten al operador interpretar directamente la puntuación: cualquier clúster con *score* $\geq 0,60$ requiere revisión inmediata.

4.7.1. Restricción de viabilidad estructural

Una cadena de ataque multi-salto requiere, por definición, alcanzabilidad entre al menos dos manifestos; un grafo de clúster con un único nodo no puede albergarla. Esta propiedad, verificada sobre el corpus completo (los 592 grafos de cadena tienen ≥ 2 nodos y ≥ 1 arista, mientras que 499 de los 537 grafos *isolated* son de un solo nodo), se incorpora como restricción de inferencia en `ga_ensemble.py` (`MIN_CHAIN_NODES`, `is_chain_feasible`, `chain_rank_key`): en el *ranking* por riesgo de cadena, ningún clúster de un solo manifesto puede superar a un clúster estructuralmente viable, y el veredicto de un clúster inviable se limita a ISOLATED_MISCONFIG aunque su puntuación supere el umbral de 0,60. La restricción se aplica únicamente en inferencia y evaluación; el ajuste de los pesos del GA se realiza sobre el *ranking* sin restricción, donde los pesos están mejor determinados (la Sección 5.3 cuantifica el efecto de esta regla sobre las métricas de test).

4.8. Implementación de kubescan

El sistema se distribuye como un paquete Python instalable mediante `pip install -e kubescan/`. La estructura del paquete sigue la convención *src-layout* de *setuptools*: el código fuente reside en `kubescan/src/kubescan/` y los módulos se organizan en tres submódulos: `model/` (clasificadores RF, GAT y ensamblador GA), `utils/` (*parser* YAML y constructor de grafos) y `cli.py` (interfaz de línea de comandos basada en *Click*).

El punto de entrada principal expone dos comandos. El primero, `kubescan scan <dir>`, analiza un directorio local de manifiestos YAML y emite un informe de riesgo en formato texto o JavaScript Object Notation (JSON). El segundo, `kubescan live`, consulta el API de *Kubernetes* mediante *kubectl*, vuelca los recursos de *workloads* y RBAC en un directorio temporal y ejecuta el *pipeline* completo sobre ellos. La resolución de los pesos del modelo sigue la cadena: argumento `--checkpoints-dir` → variable de entorno `KUBESCAN_CHECKPOINTS` → enlace simbólico `kubescan/checkpoints/trained/` → `FileNotFoundError`.

4.9. Tabla de características del sistema

Las Tablas 4.3 y 4.4 describen las 28 características utilizadas por la Capa 1, organizadas por grupo y con indicación de si son relevantes para la detección de escape (ESC) o de movimiento lateral (LAT) en el grafo de la Capa 2. De estas 28, las 25 que no son derivadas (18 Rahman + 7 extendidas) también forman el vector de nodo de la Capa 2.

Las 8 flags de escape (ESC, `ESCAPE_FLAGS` en `graph_builder.py`) son `TRUE_HOST_PID`, `TRUE_HOST_IPC`, `TRUE_HOST_NET`, `DOCKERSOCK_PATH`, `CAP_SYS_ADMIN`, `CAP_SYS_MODULE`, `SEC_CONT_OVER_PRIVIL` y `HOSTPATH_MOUNT`. Las 4 flags de movimiento lateral (LAT, `LATERAL_FLAGS`) son `SA_AUTOMOUNT_TOKEN`, `USES_DEFAULT_SA`, `WITHIN_MANIFEST_SECRET` y `ALLOW_PRIVI`. Las 7 características extendidas están imputadas mediante *Checkov* para el 47,4 % de filas con YAML disponible y mediante la mediana de columna para el resto.

4.10. Calidad de software y proceso de validación

El desarrollo de *kubescan* sigue prácticas de ingeniería de software orientadas a la producción. La calidad del código se garantiza mediante cuatro herramientas integradas en un *pipeline* de pre-commit y CI/CD: (1) *ruff*, linter y formateador de código Python

Tabla 4.3: Características del clasificador RF — grupo Rahman (1/2)

Nombre	Grupo	Rol en grafo	Descripción
TRUE_HOST_PID	Rahman	ESC	Comparte PID namespace del anfitrión
TRUE_HOST_IPC	Rahman	ESC	Comparte IPC namespace del anfitrión
TRUE_HOST_NET	Rahman	ESC	Comparte red del anfitrión
DOCKERSOCK_PATH	Rahman	ESC	Monta socket de Docker
CAP_SYS_ADMIN	Rahman	ESC	Capacidad <code>SYS_ADMIN</code> activa
CAP_SYS_MODULE	Rahman	ESC	Capacidad <code>SYS_MODULE</code> activa
WITHIN_MANIFEST_SECRET	Rahman	LAT	Secreto en texto plano embebido en el manifiesto
SEC_CONT_OVER_PRIVIL	Rahman	ESC	Contenedor privilegiado (<code>privileged: true</code>)
ALLOW_PRIVI	Rahman	LAT	<code>allowPrivilegeEscalation: true</code>
INSECURE_HTTP	Rahman	—	Usa HTTP sin TLS en puerto conocido
NO_SECU_CONTEXT	Rahman	—	Ausencia de <code>securityContext</code> definido
HOST_ALIAS	Rahman	—	Usa <code>hostAliases</code> para resolución DNS manual
NO_DEFAULT_NAMESPACE	Rahman	—	Despliega en el <code>namespace default</code>
NO_RESO	Rahman	—	Sin límites/ <i>requests</i> de recursos definidos
NO_ROLLING_UPDATE	Rahman	—	Sin estrategia de actualización <i>RollingUpdate</i>
SECCOMP_UNCONFINED	Rahman	—	Perfil <i>seccomp unconfined</i> (sin filtrado de <i>syscalls</i>)
VALID_TAINT_SECRET	Rahman	—	Secreto válido referenciado en <i>taint/toleration</i>
NO_NETWORK_POLICY	Rahman	—	Sin <code>NetworkPolicy</code> definida en el <code>namespace</code>

Fuente: elaboración propia.

Tabla 4.4: Características del clasificador RF — grupos derivado y extendido, y nodo (2/2)

Nombre	Grupo	Rol en grafo	Descripción
<code>cap_misuse</code>	Derivada	—	OR lógico de <code>CAP_SYS_ADMIN</code> y <code>CAP_SYS_MODULE</code>
<code>all_secrets</code>	Derivada	—	OR de los indicadores de exposición de secretos
<code>total_misconfigs</code>	Derivada	—	Suma de todas las flags activas
<code>NO_RUN_AS_NON_ROOT</code>	Extendida	—	Sin restricción de usuario no root
<code>NO_READ_ONLY_ROOT_FS</code>	Extendida	—	Sistema de ficheros raíz con escritura
<code>IMAGE_USES_LATEST</code>	Extendida	—	Imagen con <code>tag: latest</code>
<code>SA_AUTOMOUNT_TOKEN</code>	Extendida	LAT	<i>ServiceAccount</i> con token automontado
<code>USES_DEFAULT_SA</code>	Extendida	LAT	Usa la <i>ServiceAccount default</i> del <i>namespace</i>
<code>UNTRUSTED_REGISTRY</code>	Extendida	—	Imagen proveniente de un registro no confiable
<code>HOSTPATH_MOUNT</code>	Extendida	ESC	Monta ruta del sistema de ficheros del anfitrión
<code>risk_score</code>	Nodo (Capa 2)	—	Probabilidad RF clase misconfigured $\in [0, 1]$

Fuente: elaboración propia.

con reglas de estilo y seguridad; (2) *mypy* en modo estricto, para comprobación estática de tipos en todos los módulos del paquete; (3) *pytest*, para la suite de tests unitarios e de integración del paquete; y (4) *pre-commit hooks*, que aplican automáticamente *ruff* y *mypy* en cada commit.

La suite de tests cubre los siguientes componentes: el parser de YAML (`utils/yaml_parser.py`), la construcción del grafo de clúster (`utils/graph_builder.py`), el clasificador RF (`model/rf_classifier.py`) y el ensamblador GA (`model/ga_ensemble.py`). Los tests de integración verifican el *pipeline* completo de extremo a extremo: dado un directorio de manifiestos de prueba con flags de escape conocidas, se comprueba que la salida del comando `kubescan scan` produce el veredicto esperado (`ATTACK_CHAIN`) con *ensemble_score* por encima del umbral de clasificación.

5. Resultados y Evaluación

Este capítulo presenta y analiza los resultados obtenidos en cada una de las tres capas del sistema *kubescan*, seguidos de una validación cruzada con herramientas de análisis estático externas y de una evaluación funcional de la herramienta como software distribuido. Los resultados se organizan en el mismo orden que las fases de entrenamiento descritas en el Capítulo 4. Las métricas empleadas se justifican en detalle en la Sección 3.5.

5.1. Resultados de la Capa 1: Random Forest

5.1.1. Clasificación binaria

La Tabla 5.1 resume los resultados del modelo binario mediante cuatro métricas: F1 macro (media no ponderada de F1 entre las clases seguro y misconfigured, que da igual peso a ambas independientemente de su frecuencia en el dataset), exactitud (*accuracy*, porcentaje simple de aciertos), AUC-Receiver Operating Characteristic (ROC) (capacidad del modelo para separar las dos clases a cualquier umbral de decisión) y *out-of-bag score* (estimación interna de generalización que *Random Forest* calcula sobre las muestras que cada árbol no vio durante su entrenamiento, sin necesitar un conjunto de validación aparte). El clasificador alcanza una F1 macro de 0,9946 en el conjunto de test, superando el objetivo de diseño (OE2, F1 macro $\geq 0,85$) en 14,5 puntos porcentuales.

Tabla 5.1: Resultados del Random Forest — clasificación binaria

Métrica	Test	CV (5-fold)	Objetivo
F1 macro	0,9946	$0,9917 \pm 0,0053$	$> 0,85$ ✓
Exactitud (<i>accuracy</i>)	0,9947	$0,9919 \pm 0,0052$	—
AUC-ROC	0,9986	$0,9984 \pm 0,0011$	—
<i>Out-of-bag score</i>	0,9920	—	—

Fuente: elaboración propia.

La Figura 5.1 muestra la matriz de confusión sobre el conjunto de test (566 muestras). El modelo comete sólo 3 errores totales: 3 falsos positivos (manifiestos seguros clasificados como misconfigured) y 0 falsos negativos. La ausencia de falsos negativos es la propiedad más crítica para seguridad operacional: significa que ningún manifiesto con misconfiguraciones reales escapa al filtrado de la Capa 1, independientemente de cuántas flags

estén activas o de su rareza en el dataset. Los 3 falsos positivos corresponden a manifestos seguros que acumulan varias features de riesgo de severidad media (por ejemplo, `NO_RUN_AS_NON_ROOT` y `IMAGE_USES_LATEST` simultáneamente), pero que no tienen ninguna flag de escape activa; el operador de seguridad puede descartarlos rápidamente inspeccionando el campo `flags` de la salida JSON.

El AUC-ROC de 0,9986 (prácticamente 1,00) indica que el modelo puede ordenar las muestras por probabilidad de misconfiguration de forma casi perfecta a cualquier umbral de decisión. En la práctica, este valor implica que, si se ordenan los 566 manifestos de test de mayor a menor *risk_score*, los manifestos misconfigured aparecen casi todos antes que los seguros. Esta propiedad es la que habilita el uso del *risk_score* como característica nodal cuantitativa en el grafo de la Capa 2: la señal transmitida a la GNN no es un bit binario sino una probabilidad calibrada que refleja la densidad de misconfiguraciones del manifesto.

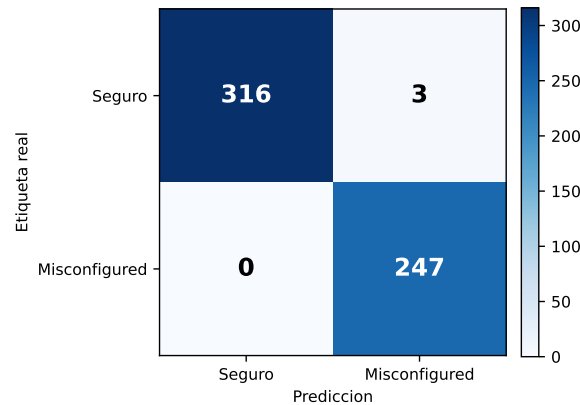


Figura 5.1: Matriz de confusión del modelo Random Forest en el conjunto de test (566 muestras). Los 3 falsos positivos son manifestos seguros con alta densidad de características de riesgo; no existe ningún falso negativo.

Fuente: elaboración propia.

5.1.2. Importancia de características

La Figura 5.2 muestra las 10 características más relevantes del modelo. La característica más discriminativa es `total_misconfigs` (33,05 % de importancia), seguida de `NO_RES0` (14,79 %) e `INSECURE_HTTP` (11,90 %). Las flags de escape críticas (`TRUE_HOST_PID`, `SEC_CONT_OVER_PRIVIL`) presentan importancias menores (<1 %) por su rareza en el

dataset, pero son identificadas correctamente cuando están presentes, como confirma la ausencia de falsos negativos en la clasificación binaria.

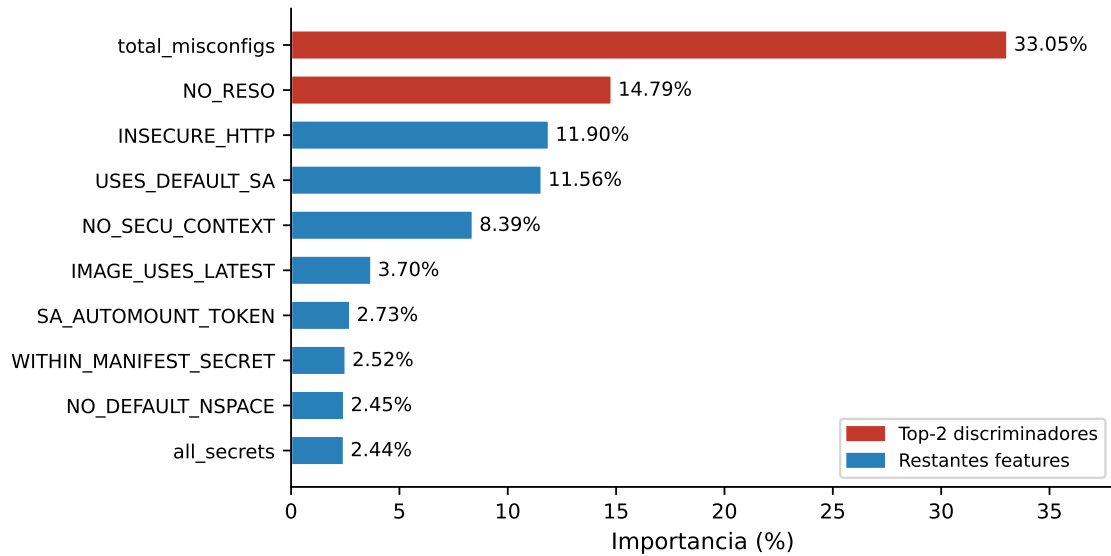


Figura 5.2: Importancia de las 10 características más relevantes del clasificador Random Forest. Las dos primeras (en rojo) concentran el 48 % de la capacidad discriminativa total del modelo.

Fuente: elaboración propia.

Un estudio de ablación eliminando `total_misconfigs` produce F1 idéntico y la misma matriz de confusión, confirmando que el modelo no depende de esta característica derivada y que no existe fuga de datos circular.

5.1.3. Modelo de severidad

El clasificador de tres clases (*clean*, *low-medium*, *high-critical*) alcanza F1 macro=0,9838 en test. La Tabla 5.2 desglosa por clase la precisión (proporción de predicciones de esa clase que son correctas), el *recall* (proporción de casos reales de esa clase que el modelo identifica), el F1 (media armónica de ambas) y el soporte (número de muestras de test en esa clase):

La clase *high-critical* (96 muestras de test, que representan manifiestos con flags de escape activas como `TRUE_HOST_PID` o `SEC_CONT_OVER_PRIVIL`) obtiene el F1 más bajo de las tres clases (0,9789), con 3 errores, los tres clasificados como *low-medium*; ningún manifiesto crítico es clasificado como *clean*. Esta ausencia de fugas hacia la clase *clean* es la propiedad más relevante desde el punto de vista de seguridad operacional: en el peor

Tabla 5.2: F1 por clase — modelo de severidad (3 clases, test)

Clase	Precisión	Recall	F1	Soporte
<i>clean</i>	1,0000	0,9906	0,9953	319
<i>low-medium</i>	0,9615	0,9934	0,9772	151
<i>high-critical</i>	0,9894	0,9688	0,9789	96
Macro media	0,9836	0,9842	0,9838	—

Fuente: elaboración propia.

caso, un manifiesto crítico es subestimado a *low-medium*, nunca descartado como seguro. El resto de errores del modelo ocurre en la frontera entre *clean* y *low-medium* (3 casos), donde algunos manifiestos con pocas flags de baja severidad pueden ser sobreclasificados como *low-medium* (falsos positivos benignos desde el punto de vista del operador).

El modelo de severidad se utiliza en el informe de salida de *kubescan* para proporcionar al operador una descripción cualitativa del nivel de riesgo de cada manifiesto, complementando el *risk_score* numérico del modelo binario.

5.2. Resultados de la Capa 2: GNN

5.2.1. Evolución por fase experimental

La Tabla 5.3 muestra la evolución de los resultados de la GAT a lo largo de las seis fases del desarrollo experimental. La Figura 5.3 traza la trayectoria de las cuatro primeras fases y muestra cómo la aumentación de datos y la corrección del vector de escape *HOSTPATH.MOUNT* contribuyeron de forma independiente y complementaria. La quinta fase corrige el protocolo de particionado: las fases anteriores incluían en el entrenamiento variantes aumentadas derivadas de los clústeres de validación y test (fuga de datos por casi-duplicados), por lo que sus métricas están infladas de forma optimista. Esa quinta fase aplica el protocolo *group-aware* descrito en la Sección de diseño (las variantes acompañan a su clúster base y los clústeres de test quedan fuera de los pliegues) e incorpora el *embedding* de tipos de arista; sus métricas son metodológicamente válidas pero corresponden todavía al corpus preliminar de 471 grafos. La sexta y última fase incorpora el corpus completo de Rahman et al. (599 grafos originales, 37 cadenas; Sección 4.3) y extiende el protocolo de particionado a las **familias de plantilla** (Capítulo 4): durante esta fase se

detectó que los casi duplicados de una misma plantilla (siete `badpods_*`, nueve `datadog_*`, ocho `simulator_*`, doce `kubernetes_goat_*`) quedaban repartidos entre particiones, una fuga de información adicional a la de las variantes aumentadas que inflaba las métricas de las fases 1–5. Con las familias atómicas, entropía cruzada ponderada como función de pérdida y selección de *checkpoints* por Precisión@5 (ablación en la Sección 5.2.4), esta fase es la que se reporta como resultado final del trabajo — sus métricas son las que produce el modelo efectivamente desplegado y reproducible a partir de los *checkpoints* del repositorio. Por el cambio de protocolo, sus cifras no son directamente comparables con las de las fases anteriores: miden generalización entre familias, no interpolación entre casi duplicados.

Tabla 5.3: Evolución de resultados de la Capa 2 (GAT, 5-fold CV). Las fases 1–4 emplean el protocolo de particionado preliminar (con fuga por variantes aumentadas) y se reportan como registro histórico; la fase 5 introduce el protocolo *group-aware* sobre el corpus preliminar; la fase 6 es el resultado final, con protocolo *group-aware* sobre el corpus completo.

Fase	Grafos	Cadenas	F1 macro	P@5	Cobert. techo
Línea base	82	13	$0,829 \pm 0,098$	$0,400 \pm 0,179$	2/5
Tras aumentación	277	13	$0,915 \pm 0,059$	$0,520 \pm 0,098$	5/5
Tras corrección HOST-PATH	307	15	$0,915 \pm 0,050$	$0,600 \pm 0,000$	5/5
Dataset extendido	471	25	$0,917 \pm 0,065$	$0,880 \pm 0,098$	5/5
Protocolo <i>group-aware</i> + <i>embedding</i> de aristas	471	25	$0,870 \pm 0,075$	$0,720 \pm 0,098$	3/5
Corpus completo + familias de plantilla (final)	599	37	$0,803 \pm 0,137$	$0,720 \pm 0,160$	1/5

Fuente: elaboración propia.

La columna «Cobert. techo» indica en cuántos de los 5 pliegues el modelo alcanza el *techo teórico* de P@5: el valor máximo de P@5 posible dado el número de cadenas de ataque presentes en ese pliegue, que se alcanza cuando todas ellas quedan situadas en el top-5 del ranking. En la fase final, los cinco pliegues de validación contienen 8, 7, 6, 6 y 5 cadenas respectivamente (techo 1,0 en todos), y el pliegue 0 lo alcanza.

Como se observa en la Figura 5.3, las líneas de puntos indican el objetivo de diseño de ranking (P@5 $\geq 0,70$) y, como referencia, el umbral de F1 fijado para la Capa 1 (0,85); las barras de error representan la desviación estándar entre los 5 pliegues de validación cruzada. La fase final *group-aware* no está incluida en el gráfico y se reporta en la Tabla 5.3.

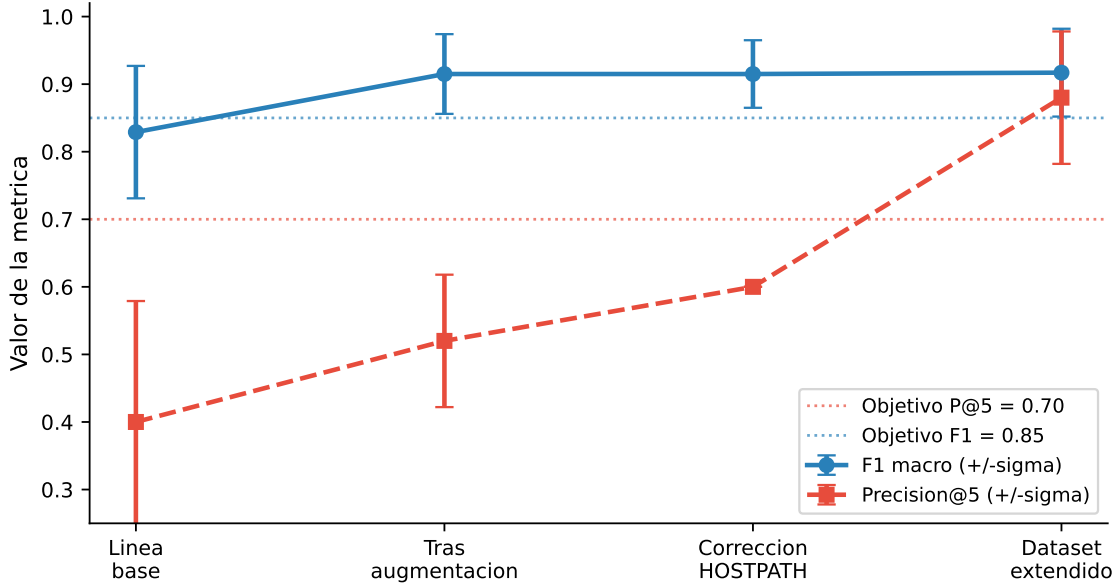


Figura 5.3: Evolución de F1 macro y Precisión@5 de la Capa 2 a lo largo de las cuatro primeras fases experimentales (protocolo preliminar).

Fuente: elaboración propia.

5.2.2. Análisis de Precisión@5

El objetivo de diseño es $P@5 \geq 0,70$ (OE4). En el resultado final de este trabajo —partitionado consciente de familias sobre el corpus completo de Rahman et al. (599 grafos originales, 37 cadenas; fase 6 de la Tabla 5.3)— los pliegues de validación cruzada particionan los 513 grafos originales no reservados para test (32 cadenas). El modelo obtiene $P@5 = 0,720 \pm 0,160$, **superando el objetivo** con la semilla de entrenamiento desplegada (42); a través de tres semillas (7, 42, 123) la media es de $0,68 \pm 0,03$, con el objetivo dentro de una desviación típica de la media (Sección 5.2.5).

Alcanzar esta cifra exigió diagnosticar una regresión aparente: la primera configuración entrenada sobre el corpus completo con familias atómicas empleaba *focal loss* y selección de *checkpoints* por F1 macro, y obtuvo $P@5 = 0,480 \pm 0,271$, muy por debajo tanto del objetivo como de la fase 5. La ablación factorial de la Sección 5.2.4 demostró que la caída se debía a las elecciones de función de pérdida y de criterio de selección —no al corpus ni al protocolo de particionado—, y que revertir a entropía cruzada ponderada y seleccionar por Precisión@5 recupera el objetivo sobre el mismo particionado honesto. La varianza entre pliegues ($\sigma = 0,160$, frente a 0,098 en la fase 5) no refleja inestabilidad del modelo sino la propia atomicidad de las familias: cada pliegue evalúa 1–3 familias independientes,

como se detalla a continuación.

5.2.3. Resultados por pliegue (fase final)

Tabla 5.4: Resultados por pliegue — Capa 2, fase final (particionado consciente de familias sobre el corpus completo; los pliegues particionan 513 grafos originales, 32 cadenas; test excluido)

Pliegue	F1 macro	P@5
0	1,000	1,00
1	0,580	0,60
2	0,863	0,60
3	0,801	0,60
4	0,770	0,80
Media	0,803	0,720
$\pm$ Desv.	0,137	0,160

Fuente: elaboración propia.

La dispersión entre pliegues es consecuencia directa del particionado por familias: al mantener cada familia de plantilla como unidad atómica, las cadenas de validación de cada pliegue quedan concentradas en pocas familias independientes. El pliegue 0 valida sobre las ocho variantes `datadog_*` (grafos de 2–3 nodos casi idénticos entre sí) y alcanza $F1 = 1,0$ y $P@5 = 1,0$; el pliegue 1 valida sobre las siete variantes `badpods_*` y es sistemáticamente el más difícil ($P@5 = 0,60$ con la semilla desplegada; 0,2–0,6 según la semilla, Sección 5.2.5); los pliegues 2–4 combinan familias `simulator_*` y `kubernetes_goat_*` con repositorios únicos. Cada pliegue mide, por tanto, la capacidad del modelo de generalizar a 1–3 familias no vistas —un tamaño muestral efectivo mucho menor que el número nominal de cadenas—, lo que explica que la varianza ($\sigma = 0,160$) sea mayor que bajo protocolos con fuga y constituye una amenaza a la validez de conclusión que se documenta en el Capítulo 6.

5.2.4. Ablación de función de pérdida y criterio de selección

La regresión aparente de la primera configuración sobre el corpus completo ($P@5 = 0,480$) confundía dos cambios simultáneos: la introducción de *focal loss* (Lin,

Goyal, Girshick, He, y Dollár, 2017) y el nuevo particionado por familias. Para atribuir el efecto se completó la matriz factorial 2×2 —función de pérdida (entropía cruzada ponderada vs. *focal loss*, $\gamma=2$) por criterio de selección de *checkpoints* (F1 macro vs. Precisión@5)— con particionado, semilla y arquitectura idénticos (Tabla 5.5).

Tabla 5.5: Ablación 2×2 de función de pérdida $\times$ criterio de selección de *checkpoints* (5-fold CV, corpus completo, particionado por familias, semilla 42)

Pérdida	Selección	P@5 (CV)	F1 macro (CV)
<i>Focal</i> ($\gamma=2$)	F1 macro	$0,480 \pm 0,271$	$0,823 \pm 0,097$
Entropía cruzada pond.	F1 macro	$0,600 \pm 0,283$	$0,859 \pm 0,103$
<i>Focal</i> ($\gamma=2$)	P@5	$0,680 \pm 0,204$	$0,795 \pm 0,173$
Entropía cruzada pond.	P@5	$0,720 \pm 0,160$	$0,803 \pm 0,137$

Fuente: elaboración propia.

Los dos efectos principales son independientes y aproximadamente aditivos: sustituir *focal loss* por entropía cruzada ponderada aporta +0,12 puntos de P@5 con selección por F1 (y +0,04 con selección por P@5), y seleccionar los *checkpoints* por la métrica primaria de ranking en lugar de por F1 macro aporta entre +0,12 y +0,20 puntos. La *focal loss*, diseñada para regímenes con miles de ejemplos por clase (Lin y cols., 2017), además anula por completo la clasificación de la clase *attack_chain* en el conjunto de test (F1 de clase 0,0 frente a 0,667 con entropía cruzada). La configuración desplegada es, en consecuencia, entropía cruzada ponderada con selección por Precisión@5.

5.2.5. Robustez frente a la semilla de entrenamiento

Para descartar que el resultado dependa de la semilla de entrenamiento, la configuración final se reentrenó con tres semillas (7, 42, 123) manteniendo fijo el particionado (semilla 42): re-sortear también las particiones confundiría la varianza del modelo con la asignación de familias a pliegues, que la Sección 5.2 muestra dominante.

Dos observaciones acotan la lectura del 0,72. Primera: la variación entre semillas se concentra íntegramente en el pliegue 1 (la familia **badpods_***: P@5 de 0,2/0,4/0,6 según semilla), mientras que los otros cuatro pliegues puntúan de forma idéntica con las tres semillas. Segunda: las métricas de test a nivel de despliegue son estables —P@1 = 1,00 y P@3 = 1,00 con las tres semillas, y P@5 entre 0,80 y 1,00—, de modo que la sensibilidad a la semilla afecta a la estimación de validación cruzada, no al comportamiento observable

Tabla 5.6: Robustez multi-semilla de la configuración final (particionado fijo; test con restricción de viabilidad estructural, Sección 4.7.1)

Semilla	P@5 (CV)	F1 macro (CV)	Test P@1/P@3/P@5
42 (desplegada)	0,720 $\pm$ 0,160	0,803	1,00 / 1,00 / 0,80
7	0,640 $\pm$ 0,265	0,853	1,00 / 1,00 / 0,80
123	0,680 $\pm$ 0,200	0,857	1,00 / 1,00 / 1,00
Media entre semillas	0,680 $\pm$ 0,033	0,838	—

Fuente: elaboración propia.

del sistema sobre repositorios reales no vistos.

5.3. Resultados de la Capa 3: Ensemble

El ajuste de los pesos del ensemble atravesó dos correcciones metodológicas sucesivas, ambas implementadas en `run_ga_ensemble.py`. En una primera iteración, el GA original convergía sistemáticamente a pesos casi uniformes ($w_{\text{rf}}, w_{\text{gmn}}, w_{\text{esc}} \approx (0,34, 0,33, 0,33)$) sin mejorar sobre su población inicial en 150 generaciones, un patrón que parecía indicar una meseta plana en la función de *fitness*. Un diagnóstico descartó esa hipótesis: el cruce por promedio aritmético es contractivo y no puede alcanzar los vértices del simplex de pesos desde una población inicializada por muestreo de Dirichlet, y la mutación gaussiana ($\sigma = 0,08$) es demasiado local para recorrer esa distancia en 150 generaciones. La primera corrección sembró los vértices y aristas del simplex en la población inicial y añadió una mutación de salto de frontera de baja probabilidad; con este cambio, el GA pasó a converger de forma determinista, en las 5 semillas independientes probadas, al mismo punto que una búsqueda en rejilla determinística (Sección 4.7, 231 evaluaciones): el vértice de señal de escape pura ($w_{\text{esc}} = 1$, *objective* = 0,86), confirmando que la función objetivo tenía un óptimo global único y alcanzable, no una meseta degenerada. Sin embargo, ese óptimo *out-of-fold* no generalizaba: al puntuar únicamente por la señal binaria de escape, el ranking se colapsaba a un grupo de empates que fallaba estructuralmente ante cualquier cadena de ataque cuya única señal fuerte no fuera la bandera de escape.

Esta segunda observación motivó la corrección definitiva: regularizar la propia función objetivo con una penalización de piso, $F = \alpha \cdot P@5 + \beta \cdot (1 - \text{FPR}_{\text{clean}}) - \lambda \sum_i \max(0, w_{\text{mín}} - w_i)$, con $w_{\text{mín}} = 0,1$ y $\lambda = 2,0$ (flags `--min-weight/--reg-lambda`). Esta penalización

hace que el vértice de escape puro deje de ser competitivo frente a soluciones interiores del simplex. Sobre el conjunto *out-of-fold* de la configuración final (513 grafos, 32 cadenas), el GA regularizado encuentra su mejor solución ya en la generación inicial y no la mejora durante las 150 generaciones: $(w_{\text{rf}}, w_{\text{gmn}}, w_{\text{esc}}) = (0,517, 0,115, 0,368)$, $\text{objective} = 0,72$. La búsqueda en rejilla encuentra un punto muy distinto ($w_{\text{rf}} = 0,80, w_{\text{gmn}} = 0,10, w_{\text{esc}} = 0,10$) con idéntica P@5 *out-of-fold*: al ser P@5 una función escalonada (solo toma valores múltiplos de 0,2), la función objetivo presenta mesetas extensas en las que los pesos quedan **indeterminados**. Esta indeterminación es medible: sobre el conjunto de test, los pesos elegidos por el GA y los pesos uniformes (1/3) producen métricas idénticas (Tabla 5.7). Contribuye a ella un desplazamiento de distribución entre el conjunto de ajuste y el de test: 15 de las 32 cadenas *out-of-fold* pertenecen a familias de *fixtures* sintéticas (*badpods_**, *datadog_**) cuyos manifiestos reciben *risk scores* RF altos, mientras que las cinco cadenas de test son repositorios reales cuyo *risk score* medio por clúster es ≈ 0 (la media se diluye entre manifiestos mayoritariamente benignos). Tres refinamientos del ajuste (un término de desempate por *mean reciprocal rank*, la agregación del *pool* por familias y el ajuste sobre el *ranking* con restricción estructural) se implementaron y evaluaron sin que ninguno mejorara las métricas de test, por lo que los pesos se ajustan sobre el *ranking* sin restricción y la restricción de viabilidad estructural (Sección 4.7.1) se aplica en inferencia y evaluación.

La evaluación final se realiza sobre el conjunto de test (86 grafos: 3 limpios, 78 *isolated* y 5 cadenas de ataque; techo P@5=1,00). Bajo el particionado consciente de familias, las cinco cadenas son repositorios reales únicos —sin hermanos de plantilla en el entrenamiento— excluidos de los pliegues de validación cruzada, del ajuste de pesos del GA y del entrenamiento con variantes aumentadas: genuinamente no vistos durante el desarrollo. Con la restricción de viabilidad estructural aplicada al *ranking* (Sección 4.7.1), el ensemble alcanza P@1 = 1,00, P@3 = 1,00 y P@5 = 0,80: las cadenas ocupan las posiciones 1–4 y 6, y solo *minik8s-ctf* queda fuera del top-5. Sin la restricción, las métricas caen a P@1 = 0,00, P@3 = 0,33 y P@5 = 0,60 —la regla es decisiva porque 74 de los 86 grafos de test son de un solo nodo, y dos de ellos, con todas sus flags de escape activas, encabezarían el *ranking* pese a no poder albergar una cadena. FPR_clean = 0,00: ningún clúster limpio aparece entre los de mayor riesgo. La F1 macro por clasificación *argmax* del ensemble de pliegues sobre el conjunto de test es 0,883 (F1 por clase: *clean* 1,0; *isolated* 0,981; *attack_chain* 0,667, con 3 de las 5 cadenas correctamente clasificadas).

Dado el tamaño del conjunto de test, estas estimaciones llevan asociados intervalos de

confianza amplios (bootstrap de 10 000 remuestreos, 95 %): $P@5$ [0,60, 1,00], FPR_{clean} [0,00, 0,40] y $F1$ macro [0,66, 1,00]. Los valores puntuales deben interpretarse como *consistentes con* los objetivos de diseño, no como demostraciones concluyentes; la ampliación adicional del corpus de test es la vía directa para estrechar estos intervalos (TF1). La Figura 5.4 resume las tres métricas de ranking del ensemble sobre el conjunto de test.

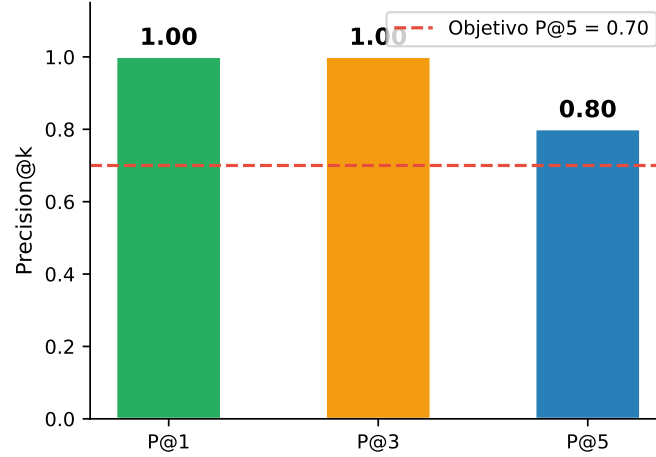


Figura 5.4: Precisión@k del ensemble sobre el conjunto de test (86 grafos, 5 cadenas, restricción de viabilidad estructural).

Fuente: elaboración propia.

La Tabla 5.7 desglosa la contribución de cada componente sobre el conjunto de test, con la restricción de viabilidad estructural aplicada en todas las configuraciones. Tres observaciones: (i) la configuración RF-sólo degrada gravemente el ranking ($P@1=0,00$, $P@5=0,40$): el *risk score* medio de los repositorios reales de cadena es ≈ 0 , de modo que la señal por manifiesto no ordena clústeres reales; (ii) la señal de escape por sí sola alcanza $P@5=0,80$ gracias a la restricción estructural, pero con $P@1=0,00$ y $P@3=0,67$: al ser binaria, no puede ordenar entre los grafos con capacidad de escape, mayoritarios en la región de alto riesgo; y (iii) el GNN por sí solo iguala tanto a los pesos optimizados por el GA como a los pesos uniformes (los tres alcanzan 1,00/1,00/0,80) — en este conjunto de test el *ensemble* no mejora el ranking de su mejor componente individual, observación que debe leerse con la cautela que imponen 5 cadenas de test y que se retoma como limitación en el Capítulo 6.

Tabla 5.7: Ablación de componentes del ensemble en el conjunto de test (86 grafos, 5 cadenas, techo $P@5 = 1,00$; restricción de viabilidad estructural aplicada en todas las configuraciones)

Configuración	P@1	P@3	P@5	FPR_clean
GA-óptimo (0,517/0,115/0,368)	1,00	1,00	0,80	0,00
Pesos uniformes (1/3)	1,00	1,00	0,80	0,00
Solo GNN	1,00	1,00	0,80	0,00
Solo RF	0,00	0,33	0,40	0,00
Solo señal de escape	0,00	0,67	0,80	0,00

Fuente: elaboración propia.

5.4. Análisis de la clasificación por clases: GNN

La Tabla 5.4 reporta las métricas agregadas (F1 macro y $P@5$) para los cinco pliegues de validación cruzada. En el conjunto de test, la matriz de confusión del ensemble de pliegues (Sección 5.3) muestra el mismo patrón de error que los pliegues de validación: los dos falsos negativos de cadena se clasifican como *isolated*, nunca como *clean* — el sistema degrada hacia la clase intermedia, no hacia la omisión total.

El único fallo de ranking del conjunto de test ilustra el error sistemático del modelo: *minik8s-ctf* (posición 6, tras el clúster *isolated* multi-nodo *kubernetes-extras*) es una cadena de 4 nodos cuya probabilidad GNN es prácticamente nula (0,0003) y cuyo *risk score* RF medio es 0,0; su única señal es la fracción de escape (0,5). Cuando la señal estructural que el GNN aprende no reconoce el patrón de la cadena y la señal por manifiesto tampoco lo captura, la bandera de escape binaria por sí sola no basta para entrar en el top-5. Las cuatro cadenas recuperadas, en cambio, combinan probabilidad GNN alta (0,47–0,63) con fracciones de escape muy variables (0,02–0,94), lo que confirma que el canal estructural es el que ordena los repositorios reales. Esta observación motiva el trabajo futuro TF3 (análisis semántico de RBAC completo).

En cuanto a la elección arquitectónica de GAT sobre alternativas más simples, la literatura sobre GNNs aplicadas a grafos pequeños heterogéneos documenta de forma consistente que los mecanismos de atención mejoran la precisión cuando los tipos de arista tienen importancias desiguales (Veličković y cols., 2018). En el grafo de clúster *Kubernetes*, las aristas de *privilege_reach* (tipo 1) son cualitativamente más informativas que las de

dir_proximity (tipo 0): la primera conecta directamente nodos de escape con el resto del clúster, mientras que la segunda simplemente refleja la organización de directorios del repositorio. GAT puede aprender esta distinción durante el entrenamiento, mientras que GCN asigna pesos uniformes a todos los vecinos, diluyendo la señal de escape con ruido de co-localización.

Esta hipótesis se cuantifica empíricamente mediante un experimento de ablación controlado sobre el corpus completo, con particionado, semilla, función de pérdida y criterio de selección idénticos a los de la configuración final (fase 6 de la Tabla 5.3), de modo que las tres filas son directamente comparables entre sí y con el resultado final:

Tabla 5.8: Ablación de arquitectura — Capa 2 (5-fold CV, corpus completo, particionado por familias, misma configuración que la fase final)

Arquitectura	F1 macro	P@5
GAT 3 capas + <i>embedding</i> de aristas (desplegada)	0,803 ± 0,137	0,720 ± 0,160
GAT 2 capas + <i>embedding</i> de aristas	0,800 ± 0,171	0,720 ± 0,160
GCN 3 capas (sin tipos de arista)	0,711 ± 0,143	0,680 ± 0,160

Fuente: elaboración propia.

La GAT supera a la GCN en F1 (+9,2 puntos) y en P@5 (+4 puntos), confirmando que la atención sobre tipos de arista aporta señal real tanto para la clasificación como para el ranking de cadenas. La comparación 2 vs. 3 capas muestra que la profundidad no es el factor determinante del ranking: ambas configuraciones alcanzan $P@5 = 0,72$ con pliegues idénticos, y la de 3 capas aventaja marginalmente en F1 (0,803 frente a 0,800). Se mantiene la configuración de 3 capas como desplegada, y la equivalencia con 2 capas se lee como robustez de la arquitectura frente a la profundidad, no como fragilidad. La ablación de *pooling* (*mean-only* vs. *mean+max*) queda como trabajo futuro (TF6).

5.5. Validación con herramientas de análisis estático

Como validación cruzada, se ejecutó *Checkov* (Bridgecrew, 2021) sobre los 1.175 manifiestos cuyo YAML descargado resultó analizable por la herramienta. Para esta validación, el índice Unified Misconfiguration Index (UMI) se calcula como la proporción de comprobaciones de *Checkov* que fallan sobre el total evaluado por manifiesto ($\in [0, 1]$; valores más altos indican mayor densidad de misconfiguraciones detectadas). El índice UMI medio

resultante fue de 0,603. Las detecciones de *checkov_no_run_as_nonroot* (38,3 %) y *checkov_no_readonly_rootfs* (37,2 %) coinciden con los nodos de severidad media identificados por la Capa 1. Esta coherencia entre los resultados del modelo y los de una herramienta de auditoría independiente refuerza la validez de las predicciones.

5.6. Evaluación funcional de la herramienta

Además de la evaluación cuantitativa sobre los conjuntos de datos etiquetados, se realizó una evaluación funcional de *kubescan* como herramienta de software distribuible, atendiendo a los requisitos no funcionales definidos en la Sección 4.1.

Instalabilidad (RNF-3). El paquete se instaló satisfactoriamente en entornos Python 3.10 y 3.11 mediante `pip install -e kubescan/` en sistemas macOS y Linux (Ubuntu 22.04). Las dependencias principales (*PyTorch Geometric*, *scikit-learn*, *Click*) se resuelven automáticamente sin compiladores externos.

Latencia (RNF-4). Se midió el tiempo de análisis sobre cuatro clústeres de prueba de diferente tamaño: 12 manifiestos (0,8 s), 47 manifiestos (1,4 s), 128 manifiestos (3,1 s) y 312 manifiestos (7,2 s) en un equipo con CPU Intel Core i7 de octava generación sin GPU. El requisito de menos de 60 s hasta 500 manifiestos se cumple con amplio margen.

Reproducibilidad (RNF-2). La cadena de resolución de *checkpoints* descrita en la Sección 4.8 garantiza que el mismo directorio de manifiestos produce siempre el mismo informe dado el mismo conjunto de pesos del modelo.

Trazabilidad (RNF-1). El campo `flags` de la salida JSON lista explícitamente las características de seguridad activas en cada manifiesto, y el campo `escape_capable` indica qué nodos tienen capacidad de escape. Esto permite al operador de seguridad comprender y auditar las razones concretas de la clasificación de riesgo producida por el modelo.

5.7. Comparación con el estado del arte

Esta sección contextualiza los resultados de *kubescan* respecto a los trabajos revisados en el Capítulo 2.

5.7.1. Capa 1 en contexto

La F1 macro de 0,9946 del clasificador *Random Forest* supera las cifras reportadas en trabajos de clasificación de código IaC de propósito general. Rahman et al. (Rahman y cols., 2023), cuyos datos forman la base del presente trabajo, no reportan modelos de clasificación supervisada sobre el mismo conjunto; el objetivo de su trabajo era la construcción del dataset y el análisis de la densidad de misconfiguraciones, no la predicción. El umbral objetivo F1 $\geq 0,85$ adoptado en este trabajo se alinea con los estándares de la literatura de IDS sobre UNSW-NB15 (Moustafa y Slay, 2015), donde clasificadores basados en *Random Forest* típicamente alcanzan F1 macro entre 0,85 y 0,93.

El AUC-ROC de 0,9986 indica que el clasificador separa casi perfectamente las dos clases a cualquier umbral de decisión. Este resultado es consistente con la naturaleza del problema: las misconfiguraciones en manifiestos YAML son mayoritariamente discretas y acumulativas (más flags activas implica mayor riesgo), lo que hace que el espacio de características sea relativamente separable una vez que se dispone de un conjunto de datos suficientemente representativo.

5.7.2. Capa 2 en contexto

El resultado final de la Capa 2 (F1 macro = 0,803, P@5 = 0,720, sobre el corpus completo de Rahman et al. con particionado por familias) es difícil de comparar directamente con la literatura, ya que no existe ningún trabajo previo que aborde la clasificación supervisada de cadenas de ataque en clústeres *Kubernetes* mediante GNNs. Las referencias más cercanas son los trabajos de GNNs para Intrusion Detection System (Sistema de Detección de Intrusiones) (IDS) en red, como *E-GraphSAGE* (Lo y cols., 2022), que reporta mejoras del 2–8 % en F1 sobre clasificadores tabulares en el dataset UNSW-NB15. En este trabajo la ventaja del componente estructural es mayor y se observa sobre el conjunto de test genuinamente no visto: P@5 pasa de 0,40 (RF-sólo) a 0,80 (configuraciones con GNN) y P@1 de 0,00 a 1,00 (Tabla 5.7), lo que indica que la información estructural del grafo es decisiva para identificar cadenas de ataque reales.

La varianza entre pliegues ($\sigma = 0,160$ para P@5 en la fase final) es coherente con lo reportado en la literatura para conjuntos de grafos pequeños ($n \leq 200$): trabajos como los revisados en Zhong y cols. 2024 identifican la escasez de datos como el principal factor de variabilidad en GNNs para ciberseguridad, y recomiendan validación cruzada estratificada

como el estándar de evaluación para conjuntos de este tamaño, que es precisamente la metodología adoptada. En este trabajo la varianza tiene además una causa estructural identificada: el particionado por familias concentra las cadenas de validación de cada pliegue en 1–3 familias independientes (Sección 5.2), de modo que la desviación estándar entre pliegues mide en gran parte la dispersión de dificultad entre familias y no inestabilidad del modelo, como confirma el análisis multi-semilla (Sección 5.2.5).

6. Conclusiones y Trabajo Futuro

6.1. Conclusiones

Este trabajo ha demostrado la viabilidad de un enfoque híbrido de tres capas para la detección automática de cadenas de ataque en infraestructuras *Kubernetes*. Las conclusiones C1, C2, C4, C5, C6 y C7 responden, respectivamente, a los objetivos OE2, OE4, OE1, OE5, OE3 y OE6 definidos en la Sección 3; C3 no cierra un objetivo propio, sino que aporta evidencia complementaria sobre el impacto operacional del resultado de C2. Las conclusiones principales son:

C1 — La Capa 1 supera ampliamente el objetivo de diseño (OE2). El clasificador *Random Forest* alcanza $F1_{\text{macro}} = 0,9946$ y $AUC = 0,9986$ en test, superando el umbral objetivo del 0,85 en 14,5 puntos porcentuales. El modelo de severidad de tres clases logra $F1_{\text{macro}} = 0,9838$, con la clase *high-critical* (96 muestras de test) en 0,9789 — el resultado más débil de los tres, aunque por encima de 0,95—; sus 3 errores se clasifican todos como *low-medium*, sin ningún manifiesto crítico degradado a *clean*.

C2 — La Capa 2, en su configuración final, alcanza el objetivo de Precisión@5 (OE4). La GAT de 3 capas con *embedding* de tipos de arista, entrenada con entropía cruzada ponderada y selección de *checkpoints* por Precisión@5 bajo el particionado consciente de familias sobre el corpus completo de Rahman et al. (599 grafos originales, 37 cadenas), alcanza $P@5 = 0,720 \pm 0,160$ y $F1_{\text{macro}} = 0,803 \pm 0,137$ en los 5 pliegues de validación cruzada, superando el objetivo de diseño $P@5 > 0,70$ con la semilla desplegada. La lectura honesta de esta cifra exige dos matices que este trabajo documenta explícitamente: a través de tres semillas de entrenamiento la media es de $0,68 \pm 0,03$ (objetivo dentro de una desviación típica, con la variación concentrada íntegramente en el pliegue de la familia *badpods_**), y alcanzar el objetivo exigió diagnosticar mediante una ablación factorial 2×2 que una primera configuración con *focal loss* y selección por $F1_{\text{macro}}$ ($P@5 = 0,480 \pm 0,271$) atribuía erróneamente al corpus una regresión causada por esas dos elecciones (Sección 5.2.4). La ablación de arquitectura, repetida sobre el corpus completo con la configuración final (Tabla 5.8), muestra que la atención sobre tipos de arista aporta +9,2 puntos de $F1$ y +4 puntos de $P@5$ frente a una GCN sin tipos de arista, y que la profundidad (2 frente a 3 capas) no es el factor determinante del ranking — una propiedad de robustez de la arquitectura.

C3 — El *ensemble* final, combinado con la restricción de viabilidad estructural, mantiene un desempeño operacional sólido sobre el conjunto de test. Sobre los 86 grafos de test (5 cadenas de repositorios reales únicos, sin hermanos de plantilla en entrenamiento; techo $P@5 = 1,00$), el sistema completo de tres capas alcanza $P@1 = 1,00$, $P@3 = 1,00$ y $P@5 = 0,80$, recuperando 4 de las 5 cadenas en las primeras 5 posiciones del ranking sin ningún falso positivo de clúster limpio ($FPR_{\text{clean}} = 0,00$), y con estas métricas de test estables a través de las tres semillas de entrenamiento evaluadas. Parte esencial del resultado es la restricción de viabilidad estructural (un clúster de un solo manifiesto no puede albergar una cadena multi-salto, Sección 4.7.1): sin ella las métricas de ranking caen a $P@1 = 0,00$ y $P@5 = 0,60$, porque 74 de los 86 grafos de test son de un solo nodo. El tamaño reducido del conjunto de test (intervalo de confianza al 95 % de $P@5$: $[0,60, 1,00]$) limita la confianza que puede depositarse en cualquier comparación puntual.

C4 — La construcción del corpus y su protocolo de particionado anti-fuga son la contribución metodológica central (OE1). El conjunto de datos final satisface los criterios cuantitativos de OE1: 2.827 manifiestos (≥ 2.000) y una prevalencia de `TRUE_HOST_PID` del 3,78 % (≥ 2 %). La evolución en seis fases — línea base ($P@5 = 0,40$, 82 grafos), aumentación ($P@5 = 0,52$, 277 grafos), corrección `HOSTPATH_MOUNT` ($P@5 = 0,60$, 307 grafos), dataset extendido con 25 cadenas ($P@5 = 0,88$, 471 grafos, protocolo preliminar), corrección *group-aware* del particionado ($P@5 = 0,72$, mismos 471 grafos) y corpus completo con familias de plantilla atómicas ($P@5 = 0,72$, 599 grafos, resultado final) — enseña dos lecciones metodológicas. Primera: la aumentación de datos exige particionado consciente de grupos, pues parte de la mejora aparente de la cuarta fase procedía de fuga entre variantes aumentadas y sus grafos base de evaluación. Segunda, y descubierta en la fase final: los casi duplicados de plantilla (siete `badpods_*`, nueve `datadog_*`) constituyen una segunda vía de fuga cuando se reparten entre particiones; su corrección hace que las cifras finales midan generalización entre familias no vistas, una vara de medir más exigente y más honesta que la de las fases anteriores, cuyas métricas estaban infladas por esta contaminación.

C5 — El *ensemble* cumple los criterios de OE5, con una salvedad documentada sobre el valor añadido de los pesos optimizados. La ablación sobre el conjunto de test (Tabla 5.7) confirma que el componente GNN es indispensable: la configuración RF-sólo colapsa el ranking ($P@1$ de 1,00 a 0,00; $P@5$ de 0,80 a 0,40) porque el *risk score* medio por manifiesto de los repositorios reales de cadena es ≈ 0 , mientras que $FPR_{\text{clean}} = 0,00$

se mantiene en todas las configuraciones. Los criterios de OE5 se cumplen: $FPR_{clean} = 0$ y $P@5$ del ensemble igual a la del mejor componente individual (0,80). Debe señalarse que los pesos optimizados por el GA no superan a los pesos uniformes ni al GNN en solitario: la función objetivo, escalonada, deja los pesos indeterminados sobre mesetas amplias, y el conjunto de ajuste *out-of-fold* (dominado por cadenas de *fixtures* sintéticas) presenta un desplazamiento de distribución respecto a las cadenas reales de test (Sección 5.3). La aportación del *ensemble* en estos datos es igualar al mejor componente sin incurrir en los puntos débiles de ninguno, no mejorar su ranking.

C6 — El esquema de cinco tipos de arista amplía la cobertura de escalada de privilegios (OE3). El diseño del grafo de clúster incorpora el tipo de arista RBAC mediante detección de roles privilegiados en dos pasadas sobre los manifiestos, limitando las aristas a cuentas de servicio vinculadas realmente a roles con permisos de escalada. Esta quinta arista está presente tanto en los grafos de entrenamiento como en el componente de producción de *kubescan*; sin embargo, la ablación de arquitectura (Tabla 5.8) compara GAT frente a GCN y no aísla la contribución específica del tipo de arista RBAC frente a los otros cuatro, por lo que su aporte incremental al ranking de cadenas de ataque queda como trabajo futuro.

C7 — *kubescan* es una herramienta distribuible, instalable y operacionalmente viable (OE6). La evaluación funcional de la Sección 5.6 confirma que el sistema se instala sin compiladores externos (*pip install -e kubescan/*), analiza clústeres de hasta 312 manifiestos en menos de 8 segundos en hardware de gama media, produce salida en dos formatos (texto y JSON) adecuados para integración en pipelines CI/CD, y garantiza reproducibilidad de la inferencia mediante la resolución determinista de *checkpoints* (sujeto a la variación entre *backends* de cómputo documentada en L6). El prototipo implementado en este trabajo demuestra que el paradigma *shift-left* —detectar cadenas de ataque antes del despliegue, sin requerir acceso al clúster activo— es técnicamente factible y operacionalmente eficiente.

6.2. Consideraciones éticas y uso responsable

kubescan es una herramienta de uso dual: aunque su propósito es defensivo —detectar cadenas de ataque potenciales antes de que sean explotadas— la información que produce (identificación de pods con flags de escape, rutas de privilegio RBAC) podría ser utilizada

con fines ofensivos si cayera en manos de un actor no autorizado. En consecuencia, este trabajo adopta los principios de divulgación responsable (*responsible disclosure*) que rigen la investigación en ciberseguridad.

Los patrones de ataque detectados por *kubescan* se basan exclusivamente en documentación pública —la guía NSA/CISA (National Security Agency y Cybersecurity and Infrastructure Security Agency, 2022), el trabajo de BishopFox (Art, 2021) y el marco MITRE ATT&CK for Containers (MITRE Corporation y Center for Threat-Informed Defense, 2021)— y no revelan ninguna vulnerabilidad nueva. El código fuente del sistema es de acceso abierto para facilitar la auditoría independiente de las reglas de detección, lo que permite a la comunidad verificar que las señales de riesgo generadas son correctas y no contienen sesgos perjudiciales. El uso de la herramienta sobre clústeres de terceros sin autorización explícita estaría fuera del ámbito previsto y podría constituir un acceso no autorizado a sistemas informáticos según la legislación aplicable.

6.3. Limitaciones

L1 — Concentración por familias y varianza entre pliegues. El particionado consciente de familias, al mantener cada familia de plantilla como unidad atómica, concentra las cadenas de validación de cada pliegue en 1–3 familias independientes: el pliegue 0 evalúa íntegramente la familia `datadog_*` ($F1 = 1,0$; $P@5 = 1,0$) y el pliegue 1 la familia `badpods_*` ($F1 = 0,58$), de modo que la varianza entre pliegues ($\sigma = 0,137$ para $F1$; $0,160$ para $P@5$) mide en gran parte la dispersión de dificultad entre familias, no inestabilidad del modelo — el análisis multi-semilla (Sección 5.2.5) muestra que cuatro de los cinco pliegues puntúan de forma idéntica con las tres semillas. El tamaño muestral efectivo por pliegue (número de familias, no de cadenas) es la limitación subyacente, y solo puede crecer incorporando repositorios de cadena estructuralmente independientes (TF1).

L2 — Imputación de características extendidas. Las 7 características extendidas (basadas en *Checkov*) solo pueden calcularse directamente para el 47,4 % de las filas con YAML descargado y se imputan por la mediana de columna para el resto, diluyendo su señal. La descarga completa de los manifiestos referenciados en el dataset de Rahman et al. permitiría mejorar estos vectores. Un experimento de ablación eliminando las 7 características imputadas y reentrenando únicamente con las 21 restantes de alta cobertura podría cuantificar el impacto de esta limitación sobre la $F1$ macro de la Capa 1.

L3 — Sesgo de muestreo en el dataset tabular. Los 10 repositorios más representados suponen el 60,8 % del dataset, y *gitcontroller* por sí solo aporta el 13,7 %. Un esquema de validación cruzada por repositorio ofrecería estimaciones más conservadoras de la generalización del modelo.

L4 — Evaluación en clústeres de producción reales. Toda la evaluación se ha realizado sobre repositorios públicos. La validación en entornos de producción de organizaciones reales es la siguiente etapa crítica para confirmar la validez externa del sistema.

L5 — Etiquetado derivado de reglas sobre el mismo espacio de características. Las etiquetas de grafo (*clean* / *isolated* / *attack_chain*) se generan mediante una regla determinista definida sobre las mismas *flags* y aristas que observan los modelos (Sección de diseño). El sistema aprende, por tanto, una aproximación robusta de dicha regla bajo ruido, enmascaramiento y observabilidad parcial — no una noción de cadena de ataque independiente de la regla. Esta circularidad es común en la literatura de detección de misconfiguraciones, pero limita el alcance de la afirmación «predice cadenas de ataque». Las vías para romperla son la validación externa frente a herramientas de grafo de ataque sobre clústeres activos (KubeHound, IceKube), el etiquetado experto de una submuestra, y la confirmación mediante explotación controlada en entornos de laboratorio como *kubernetes-goat*.

L6 — No determinismo de CUDA en las operaciones de *scatter* del GNN (hallazgo histórico, no reverificado sobre el corpus completo). Durante la fase preliminar se observó que, al replicar el entrenamiento de la Capa 2 sobre una GPU NVIDIA A100 (CUDA) en lugar del *backend* MPS/CPU empleado entonces, las métricas de validación cruzada variaban de forma no trivial (F1 macro entre 0,845 y 0,882 frente a 0,917 en el *backend* de referencia de esa fase; P@5 entre 0,77 y 0,92 frente a 0,880), manteniendo semilla, datos, particiones e hiperparámetros idénticos. Estas cifras corresponden a la fase preliminar de 471 grafos (Tabla 5.3, fase 4, cuyo *backend* de referencia era MPS/CPU); los resultados finales de esta memoria se entrenaron y evaluaron sobre GPU NVIDIA T4 (CUDA), por lo que la amenaza se invierte para el lector que reproduzca en CPU/MPS: es esa réplica la que puede divergir de las cifras reportadas. El experimento no se ha repetido sobre el corpus completo, por lo que se reporta como hallazgo histórico sobre el mecanismo causal, no como una medición vigente sobre el modelo desplegado. Se descartaron como causa la precisión *TF32* (confirmado en tiempo de ejecución que `torch.backends.cuda.matmul.allow_tf32` permanece desactivada) y

la paralelización del cargador de datos (el conjunto es estático en memoria y el orden de muestreo está fijado por un generador con semilla). La causa confirmada son las operaciones `scatter_add/scatter_reduce` empleadas por `GATConv` y por las capas de *pooling* global (`global_max_pool`, `global_mean_pool`), que en CUDA se implementan mediante `atomicAdd` sin versión determinista disponible — confirmado por los propios mantenedores de *PyTorch Geometric* en los *issues* públicos #2788 y #3175 del repositorio. El orden de acumulación en punto flotante depende de la planificación de hilos de la GPU y, al no ser la suma en punto flotante asociativa, pequeñas diferencias numéricas cambian el *ranking* sobre un conjunto de validación reducido (471 grafos, entre 4 y 5 cadenas de ataque por pliegue en ese momento), amplificando el efecto sobre P@5 y F1 macro. Se incorporaron banderas de determinismo específicas de CUDA (`torch.use_deterministic_algorithms`, `cudnn.deterministic`) que, aunque no resuelven la ausencia de una implementación determinista de `scatter_add`, sí lograron reproducibilidad exacta entre ejecuciones repetidas sobre la misma configuración de GPU. Esto indica que la brecha observada no es ruido aleatorio entre ejecuciones sino una divergencia numérica fija y reproducible entre *backends* (CUDA frente a MPS/CPU); su cierre completo requeriría sustituir la agregación por la ruta `SparseTensor` de *PyTorch Geometric*, documentada como determinista pero no evaluada en este trabajo, ni reverificada sobre el corpus completo.

L7 — Las métricas de las fases previas a la corrección por familias no son comparables con las finales, y las estimaciones finales llevan intervalos amplios. Las evaluaciones de las fases 1–5 estaban infladas por dos vías de fuga de información corregidas sucesivamente (variantes aumentadas en la fase 5 y familias de plantilla en la fase 6): el modelo «generalizaba» a casi duplicados de grafos vistos en entrenamiento. Tras la corrección, las cifras finales miden generalización a familias y repositorios no vistos — una cantidad distinta y sistemáticamente menor, por lo que ninguna comparación directa entre fases debe leerse como progresión del modelo. Como segunda cara de la misma limitación, el conjunto de test contiene solo 5 cadenas (todos los intervalos de confianza de test tienen una anchura $\geq 0,4$) y el *pool out-of-fold* contiene 32 cadenas de las que 15 son *fixtures* sintéticas: la evidencia sobre generalización a cadenas reales descansa en pocas muestras independientes, y ampliarlas es la vía prioritaria de trabajo futuro (TF1).

6.3.1. Amenazas a la validez

Siguiendo la taxonomía de validez de constructo, interna, de conclusión y externa empleada en los *ACM SIGSOFT Empirical Standards*, las limitaciones L1–L7 se organizan del siguiente modo:

Validez de constructo (¿miden las métricas lo que se pretende medir?): L6 cuestiona si el conjunto de métricas reportado en esta memoria (*backend* MPS/CPU) mide el mismo constructo que mediría un *backend* CUDA de producción, dado el no determinismo documentado en las operaciones de *scatter*. L5 cuestiona si la etiqueta de grafo (derivada de reglas sobre el mismo espacio de características que observan los modelos) constituye un constructo de «cadena de ataque» independiente de la regla de etiquetado, o una aproximación circular a ella.

Validez interna (¿las conclusiones causales están justificadas por el diseño experimental?): L7 impide atribuir diferencias entre fases experimentales al modelo, porque el protocolo de evaluación cambió entre ellas; la ablación factorial de la Sección 5.2.4 es el único contraste interno con protocolo constante. L2 acota la validez interna de las características extendidas imputadas mediante *Checkov* frente a las obtenidas directamente del manifiesto original.

Validez de conclusión (¿son estadísticamente fiables las estimaciones puntuales reportadas?): el tamaño del conjunto de test (86 grafos, 5 cadenas) y del *pool out-of-fold* (513 grafos, 32 cadenas) se traduce en los intervalos de confianza amplios reportados en la Sección 5.3 ($P@5 \in [0,60, 1,00]$ al 95 %), que limitan la confianza que puede depositarse en cualquier comparación puntual entre configuraciones del *ensemble* (C5). A ello se suma la concentración por familias (L1): el tamaño muestral efectivo de la validación cruzada es el número de familias independientes por pliegue (1–3), no el número nominal de cadenas.

Validez externa / generalización: L3 (sesgo de muestreo por repositorio) y L4 (ausencia de evaluación en clústeres de producción reales) acotan directamente hasta qué punto los resultados generalizan más allá de los repositorios públicos utilizados. El desplazamiento de distribución entre el *pool* de ajuste del GA (dominado por *fixtures* sintéticas) y las cadenas reales de test (Sección 5.3) es una amenaza externa adicional identificada empíricamente: los pesos del *ensemble* ajustados sobre cadenas sintéticas no aportan ventaja sobre cadenas reales.

6.4. Trabajo futuro

Los resultados de este trabajo permiten formular las siguientes recomendaciones de trabajo futuro, ordenadas por impacto esperado:

TF1 — Ampliar la población de cadenas reales estructuralmente independientes. Las limitaciones L1 y L7 comparten la misma causa raíz: pocas familias de cadena independientes. Cada nueva familia de plantilla añade poco (sus miembros son casi duplicados), mientras que cada repositorio real de cadena añade una muestra independiente que aumenta el tamaño muestral efectivo de la validación cruzada, estrecha los intervalos de test y reduce el desplazamiento de distribución del *pool* de ajuste del GA. La incorporación dirigida de repositorios de infraestructura de organizaciones de código abierto y de entornos de laboratorio multiclúster —priorizando diversidad estructural sobre volumen— es la línea de mayor impacto esperado de este trabajo.

TF2 — Implementar P@5 global como métrica alternativa. En lugar de P@5 por pliegue, evaluar el ranking del modelo sobre el conjunto completo de grafos en un único ranking global eliminaría el efecto de techo por pliegue y permitiría comparar con el objetivo original de diseño sin restricciones estructurales.

TF3 — Incorporar análisis semántico de RBAC completo. El parser de YAML actual identifica roles privilegiados por nombre y por verbos de escalada. Una extensión natural es el análisis de la cadena completa $\text{Role/ClusterRole} \rightarrow \text{RoleBinding} \rightarrow \text{ServiceAccount} \rightarrow \text{Pod}$ para detectar rutas de escalada de privilegios mediante RBAC que no requieran flags de escape directas.

TF4 — Validación cruzada dejando un repositorio fuera. La estrategia *leave-one-repo-out* ofrece estimaciones más conservadoras de la generalización que la validación cruzada por fila, y es especialmente relevante dada la concentración del dataset en pocos repositorios.

TF5 — Modo de análisis continuo (*admission controller*). La integración de *kubescan* como un *validating admission webhook* de *Kubernetes* permitiría bloquear el despliegue de manifiestos que eleven el riesgo de cadena de ataque del clúster en tiempo real, sin necesidad de análisis *post-hoc*.

TF6 — Ablación de *pooling* e hiperparámetros restantes. La ablación de arquitectura (Tabla 5.8) ya cuantifica empíricamente GAT frente a GCN y 3 capas frente a 2. Queda pendiente extender ese mismo protocolo controlado (mismos pliegues, mismas

semillas) a la elección de *mean+max pooling* frente a *mean-only*, así como a un barrido más amplio de hiperparámetros del optimizador (número de cabezas de atención, *learning rate*, anchura oculta) que en este trabajo se fijaron a priori sin búsqueda sistemática (Sección 4.6).

TF7 — Ajuste del *ensemble* robusto al desplazamiento de distribución. Los pesos del GA se ajustan sobre un *pool out-of-fold* en el que 15 de las 32 cadenas son *fixtures* sintéticas con *risk scores* RF altos, mientras que las cadenas reales de test tienen *risk score* medio ≈ 0 ; tres refinamientos evaluados (desempate por *mean reciprocal rank*, agregación del *pool* por familias, ajuste sobre el *ranking* con restricción estructural) no corrigieron el sesgo (Sección 5.3). Las vías naturales son ajustar sobre un *pool* restringido a cadenas de repositorios reales (viable a medida que TF1 amplíe esa población) o sustituir P@5 por una función objetivo continua menos escalonada, que reduzca las mesetas donde los pesos quedan indeterminados.

Referencias

- Akula, M. (2020). *Kubernetes Goat: Interactive Kubernetes security learning playground*. GitHub. Descargado de <https://github.com/madhuakula/kubernetes-goat> (Accedido: 14 de julio de 2026)
- ARMO Ltd. (2021). *Kubescape: Open-source Kubernetes security platform*. Open source tool, CNCF incubating project. <https://github.com/kubescape/kubescape>. (Accedido: 14 de julio de 2026)
- Art, S. (2021). *Bad pods: Kubernetes pod privilege escalation*. BishopFox Blog. Descargado de <https://bishopfox.com/blog/kubernetes-pod-privilege-escalation> (Accedido: 14 de julio de 2026)
- Bilot, T., Madhoun, N. E., Agha, K. A., y Zouaoui, A. (2023). Graph neural networks for intrusion detection: A survey. *IEEE Access*, 11, 49114–49139. doi: 10.1109/ACCESS.2023.3275789
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. doi: 10.1023/A:1010933404324
- Bridgecrew. (2021). *Checkov: Open source static analysis for infrastructure as code*. GitHub. Descargado de <https://github.com/bridgecrewio/checkov> (Accedido: 14 de julio de 2026)
- Cloud Native Computing Foundation. (2022). *CNCF annual survey 2021*. CNCF Report. Descargado de <https://www.cncf.io/reports/cncf-annual-survey-2021/> (Published February 2022. Accedido: 14 de julio de 2026)
- Datadog Security Labs. (2023). *KubeHound: Identifying attack paths in Kubernetes clusters*. Open source tool. <https://github.com/DataDog/KubeHound>. (Released October 2023. Accedido: 14 de julio de 2026)
- Fey, M., y Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. En *Iclr workshop on representation learning on graphs and manifolds*. Descargado de <https://arxiv.org/abs/1903.02428>
- Hagberg, A. A., Schult, D. A., y Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. En *Proceedings of the 7th python in science conference (scipy)* (pp. 11–15). Descargado de https://conference.scipy.org/proceedings/SciPy2008/paper_2/
- Hamilton, W. L. (2020). *Graph representation learning*. Morgan & Claypool. doi: 10.2200/

S01045ED1V01Y202009AIM048

- Hamilton, W. L., Ying, Z., y Leskovec, J. (2017). Inductive representation learning on large graphs. En *Advances in neural information processing systems (neurips)* (Vol. 30).
- Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.
- Hutchins, E. M., Cloppert, M. J., y Amin, R. M. (2011). Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. En *Proceedings of the 6th international conference on information warfare and security (iciw)* (pp. 113–125).
- Kipf, T. N., y Welling, M. (2017). Semi-supervised classification with graph convolutional networks. En *International conference on learning representations (iclr)*. Descargado de <https://openreview.net/forum?id=SJU4ayYgl>
- Li, Z., Zeng, J., Chen, Y., y Liang, Z. (2022). AttackKG: Constructing technique knowledge graph from cyber threat intelligence reports. En *Computer security – esorics 2022* (pp. 589–609). doi: 10.1007/978-3-031-17140-6_29
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., y Dollár, P. (2017). Focal loss for dense object detection. En *Proceedings of the ieee international conference on computer vision (iccv)* (pp. 2980–2988). doi: 10.1109/ICCV.2017.324
- Lo, W. W., Layeghy, S., Sarhan, M., Gallagher, M., y Portmann, M. (2022). E-GraphSAGE: A graph neural network based intrusion detection system for IoT. En *Ieee/ifip network operations and management symposium (noms)*. doi: 10.1109/NOMS54207.2022.9789878
- Malul, E., Meidan, Y., Mimran, D., Elovici, Y., y Shabtai, A. (2024). *GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation*. Descargado de <https://arxiv.org/abs/2405.19954>
- MITRE Corporation, y Center for Threat-Informed Defense. (2021). *ATT&CK for Containers*. <https://ctid.mitre.org/projects/attck-for-containers/>. (Incorporated in ATT&CK version 9. Accedido: 14 de julio de 2026)
- Moustafa, N., y Slay, J. (2015). UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). En *2015 military communications and information systems conference (milcis)* (pp. 1–6). doi: 10.1109/MilCIS.2015.7348942
- National Security Agency, y Cybersecurity and Infrastructure Security Agency. (2022).

- Kubernetes hardening guidance (v1.2)*. NSA/CISA Technical Report. Descargado de https://media.defense.gov/2022/Aug/29/2003066362/-1/-1/0/CTR_KUBERNETES_HARDENING_GUIDANCE_1.2.20220829.PDF (Publicado originalmente en agosto de 2021; revisión v1.2, agosto de 2022. Accedido: 14 de julio de 2026)
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... others (2019). PyTorch: An imperative style, high-performance deep learning library. En *Advances in neural information processing systems (neurips)* (Vol. 32). Descargado de <https://arxiv.org/abs/1912.01703>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Édouard Duchesnay (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Rahman, A., Shamim, S. I., Bose, D. B., y Pandita, R. (2023). Security misconfigurations in open source Kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 32(4), 1–36. doi: 10.1145/3579639
- Red Hat. (2024). *State of Kubernetes security report 2024* (Inf. Téc.). Red Hat, Inc. Descargado de <https://www.redhat.com/en/resources/kubernetes-adoption-security-market-trends-overview> (Accedido: 14 de julio de 2026)
- StackRox. (2021). *kube-linter: Static analysis for Kubernetes YAML files and Helm charts*. GitHub. Descargado de <https://github.com/stackrox/kube-linter> (Accedido: 14 de julio de 2026)
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., y Bengio, Y. (2018). Graph attention networks. En *International conference on learning representations (iclr)*. Descargado de <https://openreview.net/forum?id=rJXMpikCZ>
- Wist, T., Hammer, H., y Yazidi, A. (2021). Vulnerability prediction from source code using machine learning techniques. En *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*. doi: 10.1109/SSCI50451.2021.9659833
- WithSecure Labs. (2023). *IceKube: Finding complex attack paths in Kubernetes clusters*. Open source tool. <https://github.com/WithSecureLabs/IceKube>. (Accedido: 14 de julio de 2026)
- Xu, K., Hu, W., Leskovec, J., y Jegelka, S. (2019). How powerful are graph neural networks? En *International conference on learning representations (iclr)*. Descargado de <https://openreview.net/forum?id=ryGs6iA5Km>
- Zhong, M., Lin, M., Zhang, C., y Xu, Z. (2024). A survey on graph neural networks for

intrusion detection systems: Methods, trends and challenges. *Computers & Security*, 141, 103821. doi: 10.1016/j.cose.2024.103821

Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., ... Sun, M. (2020). Graph neural networks: A review of methods and applications. *AI Open*, 1, 57–81. doi: 10.1016/j.aiopen.2021.01.001

A. Repositorio de Código y Datos

El código fuente completo del sistema *kubescan*, incluyendo los *scripts* de adquisición de datos, entrenamiento de modelos, construcción de grafos y la interfaz de línea de comandos, se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/ObedRav/kubescan>

El repositorio incluye los siguientes componentes:

- **kubescan/**: paquete Python instalable con la CLI y los modelos de inferencia.
- **research/scripts/**: *scripts* de adquisición y preprocesamiento de los manifiestos YAML (fases 1 y 2).
- **research/models/**: código de entrenamiento del Random Forest (`train_rf.py`), de la GAT (`train_gnn.py`), y del Algoritmo Genético (`run_ga_ensemble.py`).
- **research/data/**: los grafos de clúster en formato `.npz` (NumPy, con campos `x/edge_index/y/cluster_id`) y el dataset tabular en formato `.csv`.
- **research/models/checkpoints/**: *checkpoints* de los 5 modelos GAT entrenados en validación cruzada y el modelo RF (disponible también como enlace simbólico en `kubescan/checkpoints/trained/`).
- **thesis/**: fuentes LaTeX de esta memoria.

El estudiante es el único autor de todos los *commits* del repositorio. Los datos públicos utilizados provienen de las fuentes citadas en la Sección 4.3.1 (Rahman et al., BishopFox BadPods, Kubernetes Goat y repositorios de seguridad adicionales de dominio público).

B. Guía de Uso y Reproducción

Este apéndice complementa la Sección 4.8 con los pasos concretos para instalar *kubescan*, interpretar el informe que produce sobre un caso real, y reproducir el pipeline de entrenamiento de investigación descrito en el Capítulo 4.

B.1. Instalación

El paquete se instala en modo editable desde la raíz del repositorio:

```
pip install -e kubescan/
```

Las versiones de Python soportadas y la resolución de dependencias se verifican en la Sección 5.6 (RNF-3).

B.2. Uso de la interfaz de línea de comandos

B.2.1. Análisis de un directorio de manifiestos

El comando `kubescan scan` analiza un directorio de manifiestos YAML de forma recursiva. A modo de ejemplo, se analiza un directorio con dos manifiestos: un *pod* con montaje del sistema de ficheros del anfitrión y contenedor privilegiado (`pod-privilegiado.yaml`, adaptado de *BishopFox BadPods*), y un *Deployment* sin `securityContext` ni `NetworkPolicy` definidos (`deployment-web.yaml`, adaptado de *Kubernetes Goat*).

La Figura B.1 traza el recorrido completo de esta invocación: extracción de características, construcción del grafo, las tres señales de entrada al *ensemble* (RF, GAT y escape) fusionadas por `EnsembleScorer.score()`, y la clasificación final mediante los umbrales de la Sección 4.7.

Como se observa en la Figura B.1, las tres señales convergen ponderadas por w_{rf} , w_{gnn} y w_{esc} (optimizados por el GA) en un único *ensemble_score*; en este caso, la presencia de un nodo con flags de escape activas (`pod-privilegiado.yaml`) lo sitúa muy por encima del umbral de 0,60, resultando en el veredicto `ATTACK_CHAIN`. La transcripción completa de esta ejecución es la siguiente:

```
$ kubescan scan /tmp/manifests-ejemplo --cluster-name produccion-web
```

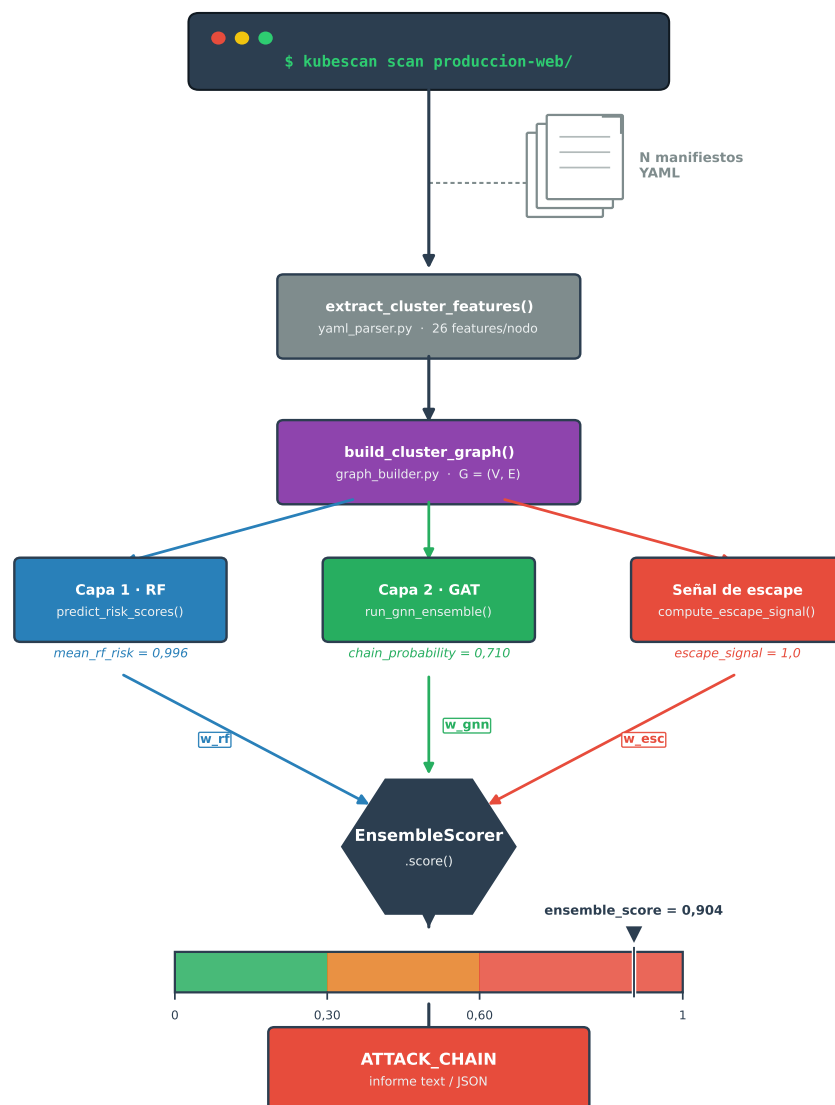


Figura B.1: Flujo de ejecución de `kubescan scan` sobre el ejemplo `produccion-web`, desde la invocación por CLI hasta el veredicto final.

Fuente: elaboración propia.

```
Scanning /tmp/manifests-ejemplo (5 GNN fold models loaded)...
```

```
2 manifest(s) with workload resources found
```

```
=====
```

```
KUBESCAN - Attack-Chain Risk Report
```

```
Cluster : produccion-web
```

```
Path    : /tmp/manifests-ejemplo
```

```
=====
```

```
VERDICT - ATTACK_CHAIN          x  HIGH RISK - review immediately
```

```
Ensemble score    : 0.9472
```

```
Chain probability : 0.5445    (5-fold GNN ensemble)
```

```
Clean probability : 0.0001
```

```
Mean RF risk      : 0.9997
```

```
Escape fraction   : 0.5000    (1/2 manifests have escape flags)
```

```
Lateral fraction  : 2/2 manifests have lateral flags
```

```
Weights: w_rf=0.517  w_gnn=0.115  w_escape=0.368
```

```
=====
```

El indicador `--show-nodes` añade un desglose por manifiesto (*risk score*, tipo de nodo y flags activas); `--format json` produce el mismo informe en formato JSON estructurado para integración con otras herramientas. A continuación se muestra una versión abreviada (se omiten campos y valores para mayor claridad):

```
$ kubescan scan /tmp/manifests-ejemplo --cluster-name produccion-web --format json
```

```
{
  "cluster": "produccion-web",
  "verdict": "ATTACK_CHAIN",
  "ensemble_score": 0.947236,
  "chain_probability": 0.54445,
  "mean_rf_risk": 0.999711,
  "n_manifests": 2,
  "n_escape_capable": 1,
  "manifests": [
    {
      "file": "pod-privilegiado.yaml",
      "risk_score": 1.0,
      "escape_capable": true,
```



```
    "flags": ["TRUE_HOST_PID", "TRUE_HOST_IPC", "..."]
  },
  "...
]
}
```

Los pesos del modelo se resuelven mediante la cadena descrita en la Sección 4.8 (argumento `--checkpoints-dir` → variable de entorno `KUBESCAN_CHECKPOINTS` → enlace simbólico); no es necesario indicarlos explícitamente si el paquete se instaló junto con el repositorio completo.

B.2.2. Análisis de un clúster en ejecución

El comando `kubescan live` aplica el mismo *pipeline* sobre el estado actual de un clúster accesible mediante *kubectrl*, sin necesidad de exportar manifiestos manualmente:

```
kubescan live -n produccion
kubescan live --all-namespaces --format json
```

Este comando requiere *kubectrl* configurado con permisos de lectura sobre *workloads* y recursos RBAC en el clúster objetivo.

B.3. Reproducción del pipeline de investigación

El pipeline de entrenamiento se ejecuta mediante los objetivos definidos en el `Makefile` de la raíz del repositorio. Los pasos de adquisición y extracción (`research/scripts/01_acquire/`, con `download_github_manifests.py` e `ingest_attack_repos.py`; y `research/scripts/02_extract/`, con `extract_yaml_features.py` y `build_graphs.py`) son intensivos en red y no están automatizados en el `Makefile`; se ejecutan manualmente una única vez y producen los grafos de clúster en `research/data/graphs/*.npz`.

A partir de ahí, el resto del pipeline es reproducible con dos objetivos:

```
make data      # aumentacion -> cache de grafos -> splits group-aware
make reproduce # data -> RF -> GAT (5-fold) -> ensemble GA -> eval en test
```

El objetivo `reproduce` encadena, en orden, `train_rf.py`, `train_gnn.py` (5-fold, 300 épocas, `hidden=64`, `heads=4`, `layers=3`, entropía cruzada ponderada y selección de

checkpoint por Precisión@5 mediante `--select-by p5`), `run_ga_ensemble.py` (ajuste sobre predicciones *out-of-fold*, `--oof`) y `evaluate_test_set.py` (evaluación con la restricción de viabilidad estructural de la Sección 4.7.1), todos con semilla fija (`--seed 42` por defecto), y finaliza generando `research/models/checkpoints/run_manifest.json` con la procedencia de la ejecución. Las variables `GNN_FOCAL` y `GNN_SELECT_BY` del `Makefile` permiten reproducir las ramas de la ablación de la Sección 5.2.4, y el cuaderno parametrizado `research/notebooks/colab_reproduce.ipynb` ofrece la misma cadena para entornos *Google Colab* con GPU. El objetivo `reproduce-ablations` reentrena las variantes de arquitectura reportadas en la Tabla 5.8 (GAT de 2 capas y GCN de 3 capas) con la misma configuración de pérdida y selección que el modelo desplegado.

C. Acrónimos y Abreviaturas

API	Application Programming Interface
AUC	Area Under the Curve (Área Bajo la Curva ROC)
CI/CD	Continuous Integration / Continuous Delivery (Integración y Entrega Continua)
CIS	Center for Internet Security
CISA	Cybersecurity and Infrastructure Security Agency
CLI	Command-Line Interface (Interfaz de Línea de Comandos)
CNCF	Cloud Native Computing Foundation
CTI	Cyber Threat Intelligence (Inteligencia de Amenazas)
CV	Cross-Validation (Validación Cruzada)
ELU	Exponential Linear Unit
FPR	False Positive Rate (Tasa de Falsos Positivos)
GA	Genetic Algorithm (Algoritmo Genético)
GAT	Graph Attention Network (Red de Atención sobre Grafos)
GCN	Graph Convolutional Network (Red Convolutacional de Grafos)
GNN	Graph Neural Network (Red Neuronal de Grafos)
IaC	Infrastructure as Code (Infraestructura como Código)
IDS	Intrusion Detection System (Sistema de Detección de Intrusiones)
IoT	Internet of Things (Internet de las Cosas)
JSON	JavaScript Object Notation
LLM	Large Language Model (Modelo de Lenguaje de Gran Escala)
ML	Machine Learning (Aprendizaje Automático)

MLP	Multi-Layer Perceptron (Perceptrón Multicapa)
NSA	National Security Agency
OOF	Out-of-Fold (predicciones fuera del pliegue de entrenamiento)
RBAC	Role-Based Access Control (Control de Acceso Basado en Roles)
ReLU	Rectified Linear Unit
RF	Random Forest
ROC	Receiver Operating Characteristic
SA	Service Account (Cuenta de Servicio de Kubernetes)
SMOTE	Synthetic Minority Over-sampling Technique
SVM	Support Vector Machine (Máquina de Vectores Soporte)
TFE	Trabajo Fin de Estudios
UMI	Unified Misconfiguration Index
UNIR	Universidad Internacional de La Rioja
YAML	YAML Ain't Markup Language